國 立 中 央 大 學

資 訊 工 程 學 系

碩 士 論 文

基於 FastDDS 的動態速率調整分散式通訊系統設計與實現

A Dynamic Rate Adjustment System for FastDDS-based Distributed Communication

研 究 生：李 泳 琪

指導教授：王 尉 任 博士

中 華 民 國　一百一十四　年　六　月

國立中央大學圖書館學位論文授權書

填單日期：2025/8/22　　2019.9版

授權人姓名	李泳琪	學號	111522605
系所名稱	資訊工程學系	學位類別	☑碩士　□博士
論文名稱	基於Fast DDS的動態速率調整分散式通訊系統設計與實現	指導教授	王尉任

學位論文網路公開授權

授權本人撰寫之學位論文全文電子檔：

・在「國立中央大學圖書館博碩士論文系統」.

(V)同意立即網路公開

(　)同意 於西元＿＿＿年＿＿＿月＿＿＿日網路公開

(　)不同意網路公開，原因是：＿＿＿＿＿＿＿＿＿＿

・在國家圖書館「臺灣博碩士論文知識加值系統」

(V)同意立即網路公開

(　)同意 於西元＿＿＿年＿＿＿月＿＿＿日網路公開

(　)不同意網路公開，原因是：＿＿＿＿＿＿＿＿＿＿

依著作權法規定，非專屬、無償授權國立中央大學、台灣聯合大學系統與國家圖書館，不限地域、時間與次數，以文件、錄影帶、錄音帶、光碟、微縮、數位化或其他方式將上列授權標的基於非營利目的進行重製。

學位論文紙本延後公開申請 (紙本學位論文立即公開者此欄免填)

本人撰寫之學位論文紙本因以下原因將延後公開

・延後原因

(　)已申請專利並檢附證明，專利申請案號：

(　)準備以上列論文投稿期刊

(　)涉國家機密

(　)依法不得提供，請說明：＿＿＿＿＿＿＿＿

・公開日期：西元＿＿＿＿年＿＿＿＿月＿＿＿＿日

※繳交教務處註冊組之紙本論文(送繳國家圖書館)若不立即公開，請加填「國家圖書館學位論文延後公開申請書」

研究生簽名：李泳琪　　指導教授簽名：王尉任

*本授權書請完整填寫並親筆簽名後，裝訂於論文封面之次頁。

國立中央大學碩士班研究生
論文指導教授推薦書

資訊工程學系碩士班 學系/研究所 李泳琪 研究生

所提之論文 基於FastDDS的動態速率調整分散式通訊系統設計與實現

係由本人指導撰述，同意提付審查。

指導教授 王昭化 (簽章)

114 年 6 月 23 日

1140623

國立中央大學碩士班研究生
論文口試委員審定書

資訊工程學系碩士班 學系/研究所 李泳琪 研究生

所提之論文 基於FastDDS的動態速率調整分散式通訊系統設計與實現

經由委員會審議，認定符合碩士資格標準。

學位考試委員會召集人 _____

委　　員 _____

_____

_____

_____

中　華　民　國　114　年　7　月　10　日

1140709

摘要

隨著自動駕駛和物聯網等技術的迅速發展，分散式即時通訊系統對動態負載適應性和跨平台一致性提出更高的要求。現有基於 Data Distribution Service (DDS) 的分散式系統通常使用靜態配置，無法根據實際負載動態調整發布頻率，且在異構平台部署時容易出現顯著的性能差異，尤其是在高負載條件下容易出現嚴重性能退化。

本研究實現了一個基於 FastDDS 的跨平台分散式自適應頻率控制系統，採用雙層通訊架構：DDS 負責高頻數據傳輸，HTTP RESTful API 負責控制指令傳遞，實現動態傳輸速率調整機制。藉由在 Subscriber 端即時監測資料接收率，計算後動態調整 Publisher 的傳輸速率。系統包含三個核心組件：SubscriberReporter 負責收集即時監測資料接收率與上報，透過 HTTP POST 回報至中央控制器；RateController 通過計算得到新的傳輸頻率；PublisherAdjuster 執行頻率調整指令，進行實時動態調整。該系統有效地提高了系統整體效能與穩定性。

實驗在真實異構環境中進行，包含 Linux 發布者節點、Ubuntu 控制節點和 Windows 訂閱者節點。設計了全因子實驗，涵蓋 3 個負載等級 (3000/5000/7000 msg/s)、3 種 QoS 配置組合 (RELIABLE-RELIABLE、RELIABLE-BEST_EFFORT、BEST_EFFORT-BEST_EFFORT) 和 2 個接收平台，共 18 個實驗場景。

實驗結果表明，自適應頻率控制器在所有 18 個場景中都實現了正向改善，成功率達100%。整體平均改善幅度為13.22%，最大改善效果達到69.90%。在極端場景下，控制器展現出強大的系統恢復能力，能將準確率從 0.40%的幾乎完全失效狀態恢復到 56.58%。跨平台性能分析發現，Ubuntu 平台表現出較好的負載承受能力和穩定性，而 Windows 平台在 RELIABLE QoS 模式下容易出現性能不穩定，但控制器能有效緩解這種差異。

關鍵字：資料分發服務(DDS), QoS 政策

Abstract

The rapid development of technologies such as the Internet of Things (IoT), and autonomous driving has led to higher requirements for decentralized real-time communication systems in terms of dynamic load adaptability and cross-platform consistency. Current decentralized systems based on Data Distribution Service (DDS) typically use static configurations that cannot dynamically adjust the dissemination frequency according to the actual load. These systems are also prone to significant performance differences when deployed on heterogeneous platforms, especially under high load conditions.

This study presents a cross-platform, decentralized, adaptive frequency control system based on FastDDS. It adopts a two-layer communication architecture: DDS is responsible for high-frequency data transmission and the HTTP RESTful API controls instruction delivery to adjust the transmission rate dynamically. Real-time monitoring of the data reception rate on the subscriber side and subsequent calculation dynamically adjust the publisher's transmission rate. The system consists of three core components: The SubscriberReporter collects and reports performance data to the central controller via HTTP POST, the RateController calculates the new publishing frequency, and the PublisherAdjuster executes frequency adjustment commands for real-time

dynamic adjustment. This system improves the overall performance and stability.

Experiments were conducted in a real heterogeneous environment that included a Linux publisher node, an Ubuntu control node, and a Windows subscriber node. The fully factorial experiments covered three load levels (3,000, 5,000, and 7,000 messages per second), three QoS configuration combinations (RELIABLE-RELIABLE, RELIABLE-BEST_EFFORT, and BEST_EFFORT-BEST_EFFORT), and two testbeds, for a total of 18 experimental scenarios.

The experimental results show that the adaptive frequency controller improves performance in all 18 scenarios, achieving a 100% success rate. The average overall improvement is 13.22%, with a maximum improvement of 69.90%. In extreme scenarios, the controller demonstrated robust system recovery, improving system reliability from near-total failure (0.40%) to 56.58% and achieving 141x performance recovery. Cross-platform performance analysis reveals that the Ubuntu platform exhibits greater load tolerance and stability. In contrast, the Windows platform is susceptible to performance vulnerability in RELIABLE QoS mode; however, the controller effectively mitigates this difference.

Keywords: Data distribution service(DDS), QoS policy

Content

Table content

Figure content

Chapter I. Introduction

1-1 Background

Data Distribution Service (DDS) is a data distribution service standard provided by the Object Management Group (OMG). It is based on publish-subscribe model and is used to achieve efficient, reliable, and scalable data distribution in distributed real-time systems. [1] With the development of communication middleware, the common Data Distribution Service (DDS) implementations currently in use are: RTI Connext[2], eProsima Fast DDS[3], OpenDDS[4], Cyclone DDS[5], Vortex OpenSplice DDS[6] and CoreDX DDS[7]. DDS plays a vital role in many industries, such as robotics[8, 9], railway[10, 11],manufacturing[12] and agricultural[13].

DDS emphasizes data-centric and provides 22 Quality of Service (QoS) strategies to meet the needs of various distributed real-time communication applications[1]. DDS aims to achieve low latency and high throughput. If QoS is properly selected, it can improve communication performance[14]. DDS aims to achieve low latency and high throughput, Almadani et al. add DDS system in agricultural supply chain to achieve high throughput[13].

Auliya et al. defined an emulation tool to calculate DDS system reliability and throughput. They proposed an algorithm to adjust sending rate with the aim

of improving reliability. However, the mechanism required the transmission rate to be manually adjusted after each experiment based on the observed reception speed, in order to maintain appropriate throughput. This process lacks real-time capability and tends to cause significant packet loss during large-scale data transmission.[15] To address these issues, this study proposes an improved system that can automatically adjust the transmission rate on real time during operation to respond dynamically to network conditions and maintain stable data transmission performance, thereby enhancing system reliability.

1-2 Motivation

In the point-to-point model, Meng et al. proposed an adaptive frame rate (AFR) mechanism for real-time transmission applications in high definition, such as cloud gaming. The AFR mechanism dynamically adjusts the frame rate based on network fluctuations and decoder capabilities, ensuring low-latency, high-quality video transmission. Experiments have shown that the AFR mechanism can reduce queueing delay by around 7.4 times and median end-to-end delay by 34%. It has also been deployed on real cloud gaming platforms. [16]The current status is analyzed in real time through closed-loop control and lightweight AI (such as k-means), and QoS policies are automatically applied. This optimizes communication performance, controls the use of computing resources, and

improves system stability and responsiveness. The system operates more stably, even in dynamic environments, avoiding frequent manual adjustments or significant latency fluctuations.[17]

In DDS systems, a fixed publisher sending rate is usually used, which cannot adapt to dynamic load changes. Resources are wasted under low loads, and performance is significantly degraded under high loads. When a large amount of data is being transmitted，the same DDS configuration shows huge performance differences on different operating systems, increasing the complexity of system deployment and reducing system predictability and maintainability. Under high load conditions, the system is prone to significant performance degradation and cannot guarantee the availability of mission-critical applications.

1-3 Contribution

Building on our previous work, this study addresses the limitations of manually adjusting the transmission rate in DDS-based systems. We propose a real-time, cross-platform adaptive control architecture that automatically adjusts the transmission rate based on actual packet reception rates. The system comprises a monitoring module within the DDS architecture and a Subscriber Reporter to report reception speed. A central Rate Controller then dynamically adjusts the Publisher's transmission rate in response.

This architecture effectively overcomes the limitations of conventional DDS configurations that rely solely on static QoS settings by enabling real-time, feedback-driven adjustments to ensure stable communication. To validate the performance of the system under heterogeneous computing environments, we deployed our framework on Windows, Linux and Ubuntu platforms and evaluated its effectiveness under varying workloads and Quality of Service (QoS) modes (RELIABLE and BEST_EFFORT). The results confirm the robustness and flexibility of the proposed adaptive control mechanism, particularly under high-load conditions.

1-4 **Structure**

This study consists of 6 chapters. Chapter I provides an introduction, Chapter II outlines the background, Chapter III discusses related work, Chapter IV introduces the system architecture, Chapter V provides a detailed description of the experimental design and results, while Chapter VI offers a conclusion to the study.

Chapter II. Background

2-1 DDS

Data Distribution Service (DDS) is a data-oriented (data-centric) communication middleware standard that is mainly applied to meet the need for real-time, high-reliability, and scalability of decentralized systems through the publish-subscribe model to achieve decentralized data distribution. DDS choose QoS to control their communication behavior and performance. The DDS schematic is shown in Figure 1.

Aditya conducted a performance comparison of common DDS implementations, focusing on differences in latency and throughput under the same experimental conditions. He proposed a method of calculating latency by comparing the write and receive times of messages in order to analyze performance under different payload conditions. He also investigated the impact of different payloads on throughput. The results showed that, among various open-source DDS software, Fast DDS has relative advantages in latency control and throughput performance. Therefore, Fast DDS was selected as the platform for this experiment.[18]

In chapter 2-1-1, the role in DDS is introduced. In chapter 2-2-2, some DDS QoS are explained.

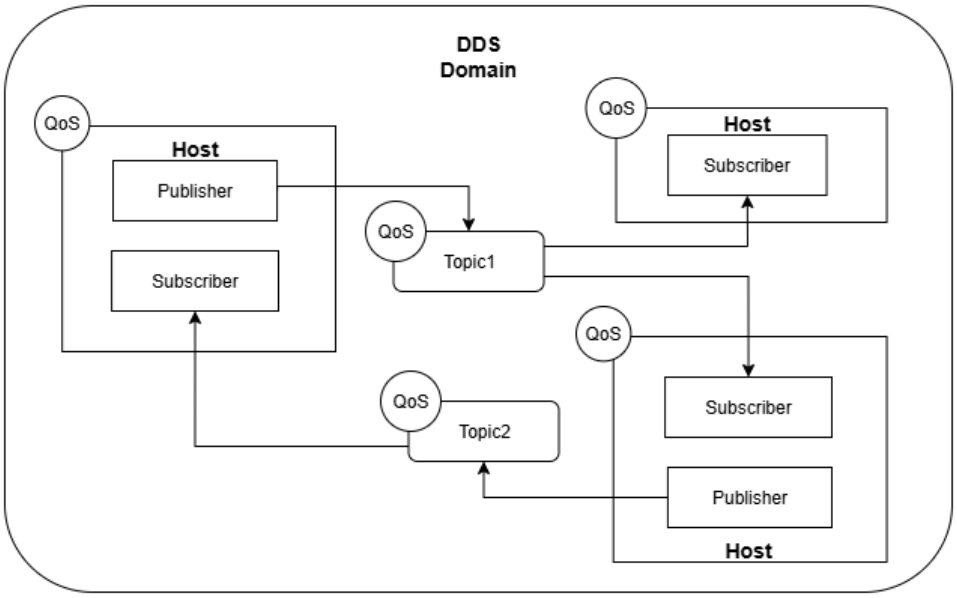

Figure 1 DDS

2-1-1 DDS introduction

The publish/subscribe architecture: Data publishers (publishers) and subscribers (subscribers) communicate through topics. This is a decentralized architecture that does not require a central agent or server; nodes can communicate directly with each other.

1. Publisher

This entity is responsible for publishing messages to multiple topics at a specified frequency. When creating this entity, the user must select a topic for data publication.

2. Subscriber

This entity is responsible for receiving and counting messages from

publishers. In order to start receiving published updates, the application needs to create a new DataReader in the Subscriber. This DataReader is then bound to the topic that describes the type of data to be received. The DataReader will then start receiving data value updates from remote publishers that match this topic.

3. Topic

Conceptually, a topic stays between the publisher and the subscriber. Each publication channel must be explicitly identified by a subscription so that the subscriber only receives the data stream they are subscribed, and not data from other publications. Topics enabling publishers and subscribers that share the same topic to match and begin communicating with each other.

2-1-2 DDS QoS

QoS policy: Users can customize detailed communication requirements (e.g., reliability, transmission priority, and data lifetime) to ensure optimal data delivery according to the application scenario.

1. ReliabilityQosPolicyKind

This QoS policy includes two types: RELIABLE_RELIABILITY_QOS and BEST_EFFORT_RELIABILITY_QOS. These types indicate the level of reliability that the service offers and requires. There is also a data member called *max_blocking_time*, which configures the maximum duration that the write operation can be blocked. The default *max_blocking_time* is 100ms. In the following tables, the term BEST_EFFORT denotes the use of BEST_EFFORT_RELIABILITY_QOS, while RELIABLE refers to RELIABLE_RELIABILITY_QOS.

Table 1 ReliabilityQosPolicyKind Compare

Different	RELIABLE_RELIABILITY_QOS	BEST_EFFORT_RELIABILITY_QOS
Data Loss	Allowed. Do not guarantee every data can be successfully transmitted or received	Not allowed. All data is transmitted and received
Retransmission	No retransmission	Retransmit until data is received successfully or timeout occurs
max_blocking_time	No effect	If the sender fails to send data within *max_blocking_time*, it will unblock.
Impact	Fast transmission rate Low latency Data loss may occur	Slow transmission rate High latency Data integrity can be ensured

There are some compatibility rules of ReliabilityQosPolicy, to maintain this in DataReaders and DataWriters, the type of DataWriter must be greater than or equal to the type of DataReader. As can be seen from the table, not all selections are compatible. The default setting for the DDS system is for DataWriter to use the RELIABLE_RELIABILITY_QOS setting and for DataReader to use the BEST_EFFORT_RELIABILITY_QOS setting. In Table 2, BEST_EFFORT represents for BEST_EFFORT_RELIABILITY_QOS, RELIABLE represents for

RELIABLE _RELIABILITY_QOS.

Table 2 ReliabilityQos compatible rules

PUBLISHER	**SUBSCRIBER**	**Match**
BEST_EFFORT	BEST_EFFORT	Yes
BEST_EFFORT	RELIABLE	No
RELIABLE	BEST_EFFORT	Yes
RELIABLE	RELIABLE	Yes

Chapter III. Related Work

Our team defined a DDS system model, which is used to calculate topic-based reliability and throughput. And they proposed an algorithm to optimize the reliability and throughput of message transmission within the DDS system by adjusting the sending rate of each publisher. Firstly, established a DDS-based system configuration, defining fundamental DDS components such as publishers and subscribers, and further specifying the concepts of sending and reception rates. Based on this foundation, the system's throughput and reliability were first defined. To calculate throughput, the transmission rate of each message is first computed for each topic, yielding the corresponding per-topic throughput. This is then aggregated to calculate the overall system throughput. For reliability, the publisher sets an initial transmission rate during message transfer. Ideally, the subscriber's reception rate would approximate the publisher's transmission rate to form an expected reception rate. However, due to network conditions and other factors, messages are lost at the subscriber, causing a discrepancy between the actual and expected reception rates. Researchers calculate the reliability of each topic by determining the rate at which messages are lost and taking into account the difference between the expected and actual rates. The overall system reliability is then derived from the per-topic reliability. Based on this, an algorithm was

designed to calculate the expected and actual reception rates for each machine and obtain the message reduction ratio by using the ratio of the two. The algorithm then selects the smallest message reduction ratio across all hosts and uses this as the basis for adjustment, specifically by multiplying the original transmission rate by this minimum reduction ratio to calculate a new transmission rate. Finally, the transmission rate of the publisher is manually adjusted and the results observed. According to the results, the algorithm single-topic reliability enhance more than 20% and single-topic has a little improve[15].

Fu et al. designed and implemented a decentralized communication architecture for a simulation system based on DDS. Experimental results show that, although the average time required to transmit data is longer in the reliable transmission mode, no packet loss occurs, ensuring the successful transmission of each data sample. In the best effort transmission mode, the transmission delay time is short but packet loss occurs. Modifying the DDS's QoS (Quality of Service) policy demonstrates that this approach enables the system to achieve high reliability and low latency. [19]

Chapter IV. Proposed Method

This chapter introduces the system architecture. Section 4-1 introduces the system architecture and describes the DDS and HTTP functions separately. Section 4-2 introduces the system model, and Section 4-3 explains the algorithm.

4-1 System Architecture

The system uses a distributed Data Distribution Service (DDS) architecture comprising three platforms, as outlined below.

Host1 is the publisher host, which includes the publisher and publisher adjuster. Its role is to continuously send data to multiple DDS Topics (Topic1 to Topic11). The PublisherAdjuster provides a RESTful API interface (HTTP, port 8000) that allows external the Controller to dynamically adjust the transmission rate of the Publisher.

Host2 and host3 is the subscriber host, which includes the subscriber and subscriber reporter. As the subscriber endpoint, it receives data published by the publisher and calculates the reception rate and packet loss for each topic. The SubscriberReporter then periodically reports the reception rate.

The Rate Controller may be deployed on any host or run independently on a separate host. For resource constraints, this study configures it to operate on

Host2. The controller periodically sends adjustment commands to the publisher adjuster based on the reception status to achieve dynamic frequency control.

Communication between hosts uses the Fast DDS middleware. The publisher and all subscribers use the DDS protocol to transmit data, while the controller and publisher adjuster use the HTTP protocol to exchange control commands.

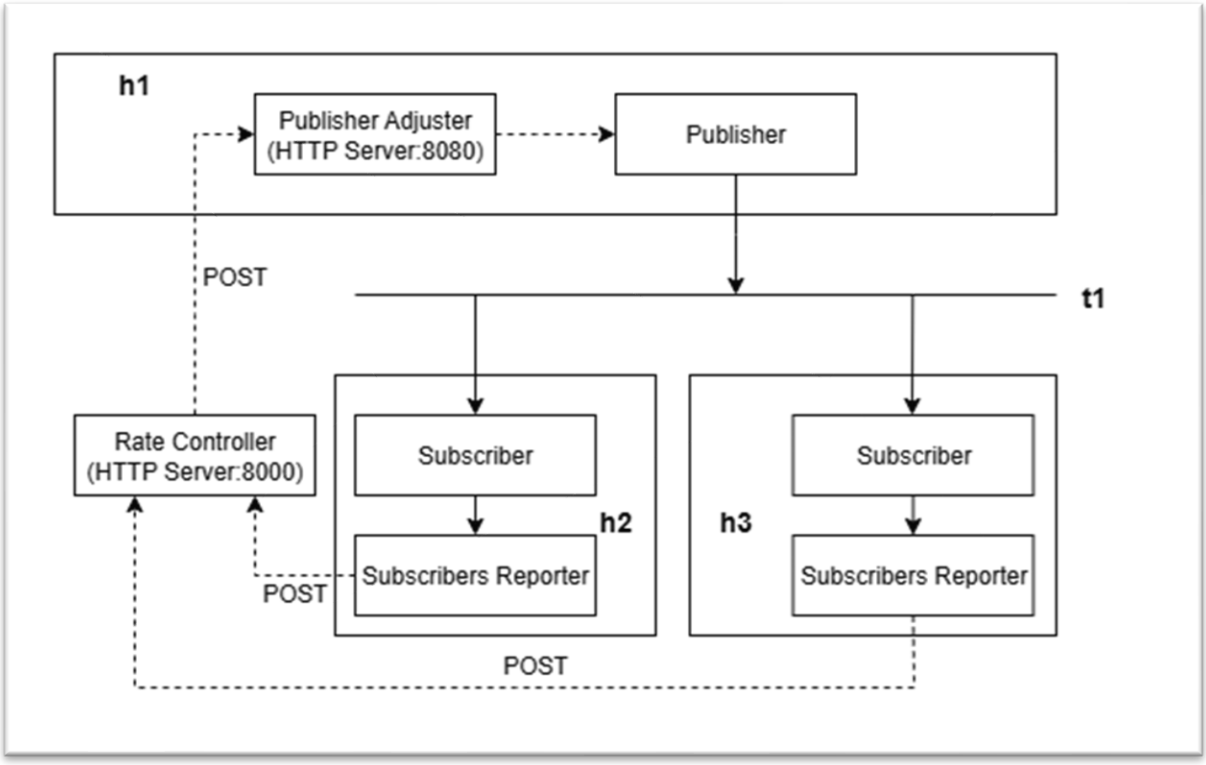

Figure 2 System Architecture

4-1-1 DDS Component

DDS component consists of three characters: publisher, subscriber and topic. Each character has a different task, which is introduced below.

1. Publisher

Responsible for sending messages to multiple topics.

2. Subscriber

Subscriber is responsible for receiving messages. Also, counting messages from publishers and get the receiving rate.

3. Topic

Receiving messages from publisher and forwarding them to different subscribers.

4-1-2 Rate Adjustment Component

Rate adjustment component including three roles: Subscribers Reporter, Publisher Adjuster, Rate controller. Details are as follows.

1. Subscribers Reporter

The Subscriber Reporter is responsible for regularly collecting subscriber performance data and reporting it to the controller. It collects data in a unified manner and centrally manages the performance metrics of multiple subscribers. Data is reported periodically, with statistical reports sent to the controller every 10 seconds. Standardized monitoring interfaces are provided across different platforms. To get started, set the host ID (host_id=1 for Ubuntu and host_id=2 for Windows), configure the controller connection parameters (8000) and start an

independent reporting thread. During runtime, each subscriber registers its rate atomic variable pointer with the Reporter. Every 10 seconds, the current receive rate of all subscribers is collected and packaged into JSON format before being sent to the controller via a POST request.

2. Publisher Adjuster

The Publisher Adjuster component is responsible for executing frequency adjustments in the system. It has a built-in HTTP server that receives adjustment commands from the controller. The HTTP server must be initialized, RESTful API endpoints registered, an independent server thread started, and adjustment log recording enabled. Each publisher registers its interval atomic variable pointer with the Adjuster. It then receives adjustment instructions from the controller via HTTP POST /adjust. The publisher verifies that the adjustment parameters are within a reasonable range and updates the transmission intervals of all publishers atomically.

3. Rate Controller

Responsibilities include collecting performance data and making frequency adjustment decisions. This involves setting the initial release rate, starting the HTTP server and initializing the HTTP client connection to Publisher Adjuster. Receive performance reports from two hosts, calculate the reception ratio for each

host, analyze the global minimum ratio, formulate adjustment strategies and implement them.

Dynamically adjusts the transmission rate under the DDS architecture and makes a system status judgment before the adjustment to ensure the rationality and stability of the adjustment behavior. Specifically, if the receiving end receives fewer than 100 msg/s, the system considers this to be a disabled state and assumes that transmission is not currently performing well. Then sets the sending rate to 100 msg/s to prevent continuous high-load transmission from wasting system resources.

Additionally, the system will check the minimum ratio to assess stability. After conducting multiple experiments, we determined that the critical value is 0.97. We chose 0.97 as the critical value to avoid making the system overly sensitive to minor fluctuations, which could lead to frequent adjustments. If the ratio falls below 0.97, it indicates system instability and triggers the controller to adjust the rate. Conversely, if the ratio exceeds 0.97, it suggests that the system is stable and does not require intervention from the controller. This ensures that the system maintains stability even under minor changes.

Three system status are described as follows:

(1) System disabled

If the observed receiving rate is smaller than 100msg/s. Then controller directly set the new sending rate to 100msg/s.

(2) System unstable

If the observed receiving rate is bigger than 100msg/s and the sending rate adjustment ratio is smaller than 0.97. Then the controller will send the new sending rate to publisher.

(3) System stable

If the observed receiving rate is bigger than 100msg/s and the sending rate adjustment ratio is bigger than 0.97.

4-2 System Model

Chapter 4-2-1 will introduce DDS component related symbols and definition. In chapter 4-2-2, it will introduce the definition of rate adjustment. Chapter 4-2-3 shows sending rate adjustment algorithm.

4-2-1 DDS component

This chapter introduce DDS component related symbols and define each topic throughput, system throughput and each topic reliability.

At first, define the DDS component symbols shown in Table 3.

Table 3 the DDS component symbols

Symbols	Explanation
D	DDS system
A	the number of publishers
B	the number of subscribers
C	the number of hosts
K	the number of topics
p_n	the *m*th publisher, *n* is from 1 to A
s_n	the *n*th subscriber, *n* is from 1 to B
h_m	the *n*th host, *m* is from 1 to C
t_j	the *j*th topic, *j* is from 1 to K
ph_{mn}	if publisher p_n is in host h_m, ph_{mn}=1 else ph_{mn}=0
pt_{jn}	if p_n subscribes to topic t_j, pt_{jn} =1 else pt_{jn} =0
sh_{mn}	if subscriber s_n is in host h_m, sh_{mn} =1 else sh_{mn}=0
st_{jn}	if s_n subscribes to topic t_j, st_{jn} =1 else st_{jn} =0
α_{jn}	the scheduled sending rate from publisher p_n to topic t_j
α'_{jn}	the adjusted sending rate from publisher p_n to topic t_j
β_{jn}	the observed receiving rate from subscriber s_n to topic t_j
ω_j	the number of subscribers subscribes to topic t_j
φ_m	the scheduled receiving rate in host h_m

Symbols	Explanation
$\varphi_m{}'$	the observed receiving rate in host h_m
$T(t_j)$	each-topic throughput of topic t_j
T_D	system throughput
$R(t_j)$	each-topic reliability of topic t_j
R_D	system reliability
sr_j	the scheduled receiving rate from topic t_j to all subscribers
srt_j	the scheduled receiving rate from topic tj to a subscriber
g_{jm}	the decrease ratio of topic t_j in host h_m
$Ratio_j$	the sending rate adjustment ratio of topic t_j
$threshold_{max}$	if $Ratio_j > threshold_{max}$, the system is stable else the system is not stable
$threshold_{min}$	if $\varphi_m{}' < threshold_{min}$, the system is disabled else the system is not stable

Then we define each-topic throughput and system throughput, as well as each-topic reliability, all of which refer to a previous study by our team.

1. Each-topic Throughput

The throughput for each topic is positive when subscriber s_n successfully subscribes to topic t_j, otherwise the throughput is 0. β_{jn} is the observed receiving rate from subscriber s_n to topic t_j.

$$T(t_j) = \sum_{n=1}^{B} \beta_{jn} \tag{1}$$

2. System Throughput

In the DDS system D, there are serval topics t_j, with the number of topics ranging from 1 to K.

$$T_D = \frac{\sum_{j=1}^{K} T(t_j)}{K} \tag{2}$$

3. Each-topic Reliability

ω_j is the number of subscribers who have subscribed to topic t_j. st_{jn} is a Boolean value, when p_n successfully subscribes to topic t_j, the value is 1; otherwise, the value is 0.

$$\omega_j = \sum_{n=1}^{B} st_{jn} \tag{3}$$

sr_j is the scheduled receiving rate from topic t_j to all subscribers. The scheduled sending rate from publisher p_n to topic t_j.

$$sr_j = \sum_{n=1}^{A} (\omega_j \cdot \alpha_{jn}) \tag{4}$$

The reliability of a topic t_j is based on its message loss rate. The loss rate is obtained by subtracting the observed receiving rate from the scheduled

receiving rate, and then dividing by the scheduled receiving rate to calculate the reliability.

$$R(t_j) = \frac{sr_j - \sum_{n=1}^{B} \beta_{jn}}{sr_j} \tag{5}$$

4. System Reliability

The DDS system D reliability is the average of K topics reliability.

$$R_D = \frac{\sum_{j=1}^{K} R(t_j)}{K}$$

4-2-2 Rate Adjustment component

In the DDS system, the topic t_j receives messages and sends them to the subscriber s_K. Ideally, the scheduled receiving rate from topic t_j to one subscriber should be equal to the scheduled sending rate α_{jn}.

$$srt_j = \sum_{n=1}^{A} \alpha_{jn} \tag{6}$$

The scheduled receiving rate in host h_m from topic t_j to subscriber s_n. st_{jn} and sh_{mn} are Boolean values. If subscriber s_n is in host h_m and s_n subscribes to topic t_j, it will be 1; otherwise, it will be 0.

$$\varphi_m = \sum_{j=1}^{K} \sum_{n=1}^{B} (srt_j \cdot st_{jn} \cdot sh_{mn}) \tag{7}$$

The observed receiving rate in host h_m from topic t_j to subscriber s_n. st_{jn} and sh_{mn} are Boolean values. If subscriber s_n is in host h_m and s_n

subscribes to topic t_j, it will be 1; otherwise, it will be 0.

$$\varphi_m{}' = \sum_{j=1}^{K}\sum_{n=1}^{B}\left(\beta_{jn}\cdot st_{jn}\cdot sh_{mn}\right) \tag{8}$$

The decrease ratio of topic t_j in host h_m. Using the scheduled receiving rate dividing by the observed receiving rate to get the ratio of each topic t_j in host h_m. The ratio shows the performance of the host h_m.

$$g_{jm} = \frac{\varphi_m}{\varphi_m{}'} \tag{9}$$

After getting the decrease ratio g_{jm}, we can find the minimize ratio $Ratio_j$, in all topic t_j from 1 to K. It shows topic t_j in host h_m has worst performance.

$$Ratio_j = min_{m=1}^{C}\{g_{jm}\} \tag{10}$$

Then we will adjust the sending rate to a new sending rate α'_{jn}. The new sending rate will be close to the real DDS environment. Firstly, we will check if the system is disabled.

$$\alpha'_{jn} = 100,\ \ if\ \varphi_m{}' < threshold_{min} \tag{11}$$

If system isn't disabled, then we will decide the system status to get new sending rate.

$$\begin{cases} \alpha'_{jn} = \alpha_{jn}, if\ Ratio_j > threshold_{max} \\ \alpha'_{jn} = \alpha_{jn} \cdot Ratio_j, otherwise \end{cases} \quad (12)$$

4-3 **Algorithm**

This chapter introduce the algorithm, it will explain how the adjustment works. First, in DDS system D, we calculate the scheduled receiving rate φ_m and the observed receiving rate $\varphi_m{}'$. The use of st_{jn} and sh_{mn} to confirm two things: firstly, that a topic t_j has been subscribed to by a subscriber s_n, and secondly, that the subscriber s_n is in a host h_m. After that we get the decrease ratio g_{jm} for each topic. Then we find the smallest ratio $Ratio_j$ of the DDS system D. Now we assess the status of DDS system D by the ratio $Ratio_j$. If $Ratio_j$ is greater than the maximum threshold $threshold_{max}$, the system is stable and the adjusted sending rate α'_{jn} is equal to the scheduled sending rate α_{jn}. If $Ratio_j$ is less than the minimize threshold $threshold_{min}$, the system is disabled and we directly set the adjusted sending rate α'_{jn} to 100msg/s to ensure the system is returns to normal. Otherwise, the system is unstable but not disabled, we calculate the adjusted sending rate α'_{jn} by the scheduled sending rate α_{jn} and the ratio $Ratio_j$. Finally, we noticed all the host h_m with the adjusted sending rate α'_{jn} .

Algorithm 1: Adjustment of the Sending Rate Algorithm

Input: D=< $h_m, t_j, st_{jn}, sh_{mn}, \alpha_{jn}, srt_j, \beta_{jn}$ >

Output: α'_{jn}

---

for each host h_m **in** D **do**

get $\varphi_m = \sum_{j=1}^{K} \sum_{n=1}^{B} (srt_j \cdot st_{jn} \cdot sh_{mn})$

get $\varphi_m{}' = \sum_{j=1}^{K} \sum_{n=1}^{B} (\beta_{jn} \cdot st_{jn} \cdot sh_{mn})$

for each topic t_j **do**

$g_{jm} = \frac{\varphi_m}{\varphi_m{}'}$

get $Ratio_j = min_{m=1}^{C} \{g_{jm}\}$

end for

if system disabled

$\alpha'_{jn} = 100$

return α'_{jn}

else if system stable

$\alpha'_{jn} = \alpha_{jn}$

return α'_{jn}

else if system unstable

$\alpha'_{jn} = \alpha_{jn} \cdot Ratio_j$

return α'_{jn}

for each host h_j **do**

for each topic t_j **do**

$\alpha_{jn} = \alpha'_{jn}$

end for

Chapter V. Experiment

This chapter will present the experiments conducted using this system. Section 5-1 will introduce the software and hardware configurations used to conduct the experiments. Section 5-2 will present the experimental design plan, and Section 5-3 will explain the experimental data results.

5-1 System configuration

In this section, we will introduce the system-related configurations used to conduct the experiments for this study. Section 5-1-1 covers hardware configuration. Section 5-1-2 covers software configuration, which includes detailed descriptions of middleware platform and communication framework.

5-1-1 Hardware Configuration

This experiment will use three hosts, host1, host2, and host3. These three hosts use different operating systems. The details are as follows.

Table 4 Hardware Configuration

Host	OS	CPU	CPU Core(s)	Memory
Host 1	Linux 5.15.0	Intel(R) Core(TM) i7-4790	4	14GB DDR3-1600MHz
Host 2	Windows 10	Intel(R) Core(TM) i7-12700	12	24GB DDR5-4000MHz

Host	OS	CPU	CPU Core(s)	Memory
Host 3	Ubuntu 22.04	Intel(R) Core(TM) i7-3770	4	24GB DDR3-1600MHz

5-1-2 Software Configuration

1. Middleware Platform

Table 5 Middleware Platform

Host	Middleware	Version	License
Host 1	Fast DDS	3.2.1	Apache License 2.0
Host 2	Fast DDS-Windows	3.2.0	Apache License 2.0
Host 3	Fast DDS	3.2.1	Apache License 2.0

5-2 Experiment Design and Process

This chapter includes experiment design and process. In chapter 5-2-1 introduce experiment design in details. In chapter 5-2-2 describes experiment process.

5-2-1 Experiment Design

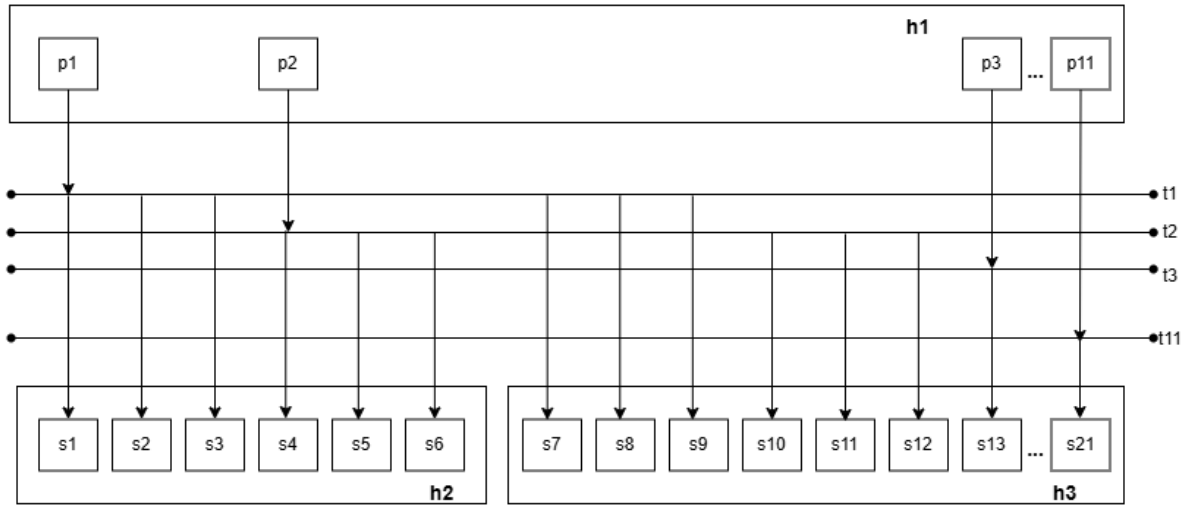

Figure 3 DDS system

The aim of this experiment is to evaluate the impact of the proposed dynamic transmission rate controller on DDS systems under different Quality of Service (QoS) settings and transmission rates. The experimental setup consists of one publisher host (Linux), two subscriber hosts (Windows and Ubuntu) and one controller (Windows), which is responsible for dynamically adjusting the publisher's transmission interval. The controller adjusts the publisher's transmission rate based on reception rate information returned by the subscribers, with the aim of improving message reception reliability.

The experiment involves three hosts: host1 is the publisher and is responsible for publishing messages under different loads. In host1, 11 publishers should be created, with each publisher publishing 1,000,000 messages and corresponding to a topic (t1 to t11). Host2 is responsible for decision-making and message subscription. 6 subscribers are created in host2, each subscribing to topics t1 and t2. Host3 creates 15 subscribers and is responsible for message subscription. It is shown in Figure 4.

The experiment has three variables: low, medium, and high workload, which are used to test the controller's adaptability under different loads shown in Table 6. Three QoS combinations are compared shown in Table 7 is to test different reliability requirements and to compare scenarios with and without a controller, in order to validate the controller's improvement effects.

To evaluate the impact of the dynamic control mechanism on the stability of the DDS system under different operating conditions, this study focuses on three main variables: workload, reliable QoS and controller. The introduction is as follows.

First, workload is set to simulate system behavior under various stress levels. The experiment includes three workload scenarios with transmission rates of 3,000 msg/s, 5,000 msg/s, and 7,000 msg/s, which respectively correspond to low,

medium, and high workload conditions shown in Table 6. This design aims to observe the system's performance and stability at various resource utilization levels, particularly its data receiving capability and packet loss on subscriber end.

Second, the reliable QoS choice considers the performance and reliability trade-offs of different scenarios. In FastDDS, RELIABLE_RELIABILITY_QOS ensures data delivery integrity, making it suitable for tasks requiring high reliability. On the other hand, BEST_EFFORT_RELIABILITY_QOS is suitable for scenarios requiring high timeliness and performance that can tolerate some data loss. To investigate the impact of QoS pairing on system performance, three QoS combinations are designed and tested according to the Fast DDS compatibility rules shown in Table 7: Plan A (both parties are Reliable), Plan B (only the publisher is Reliable), and Plan C (both parties are Best Effort).

Finally, this study compares the reliability and performance of the system under two scenarios: when the controller is not enabled (fixed transmission rate) and when it is enabled (dynamically adjusting the rate according to the reports). This evaluation aims to assess the controller's effectiveness in managing various workload and QoS conditions.

Table 6 Workload

Scenario	Sending rate(msg/s)
Low	3,000msg/s

Medium	5,000msg/s
High	7,000msg/s

Table 7 DDS Reliability QoS Policy

Plan	Publisher QoS	Subscriber QoS
A	RELIABLE	RELIABLE
B	RELIABLE	BEST_EFFORT
C	BEST_EFFORT	BEST_EFFORT

Table 8 Controller Status

Controller	Status
A	Enabled
B	Disable

5-2-1 Experiment Process

Each experiment consists of five phases. First, establish the publisher and subscriber. Then, wait for the system to reach a stable state. Next, start the controller to observe the adjustment of the transmission speed. Finally, shut down each component. The details of each phase are described below.

1. Phase 1- DDS component start-up phase(t=0s)

First, start the publisher on host 1 and the subscribers on hosts 2 and host 3 simultaneously.

2. Phase 2- System stabilization period (t=0-40 s)

Waiting for the DDS connection to be established and stabilized.

3. Phase 3- Controller component start-up phase(t=40s)

Start the controller on host 2.

4. Phase 4- Main experiment phase (t=40s to the end)

Monitor the experiment and collect the data. Host 2 and host 3 reports the receiving rate by subscriber reporter. Controller calculate new sending rate and send to publisher adjuster. Publisher adjust sending rate.

5. Phase 5- End of the experiment phase

Stop all the components in host1, host2 and host3.

5-3 Experiment Results

The experimental results will be displayed separately for each workload. For each workload, the total and expected number of messages received will be displayed, and the reliability will be calculated. The table shows the machine's performance in detail under different plans, and the difference between with and without the controller can also be seen.

5-3-1 Low workload-3000msg/s

Table 9 Host2-Low workload

		no controller		**controller**		
Plan	Expected	Received	System Reliability	Received	System Reliability	Improve
A	6,000,000	4,631,020	77.18%	5,906,755	98.45%	**21.26%**
B	6,000,000	5,754,136	95.90%	5,897,764	98.30%	**2.39%**
C	6,000,000	5,891,697	98.19%	5,894,685	98.24%	**0.05%**

Table 10 Host3-Low workload

HOST3-3000msg/s		**no controller**		**controller**		
Plan	Expected	Received	System Reliability	Received	System Reliability	Improve
A	15,000,000	14,742,172	98.28%	14,813,920	98.76%	**0.48%**
B	15,000,000	14,741,106	98.27%	14,741,282	98.28%	**0.01%**
C	15,000,000	14,741,324	98.28%	14,742,466	98.28%	**0.01%**

In host2, the reception rate under Plan A is lower than that under Plan B and Plan C because it requires more resources. When the controller is activated, the transmission speed adjusts to 2857msg/s and system acceptance reliability improves from 77.18% to 98.45%. In host3, the performance is good even without the controller, and adding the controller only results in a slight improvement.

1. Plan A

Without a controller, it can be observed that the reception rate of host2 fluctuates between 2000msg/s and 2600 msg/s, which is not very stable. Meanwhile, host3 has good reception capability and can reach 3000 msg/s. At this stage, the per-topic reliability ranges from 88% to 100%. The transmission speed is set to 2857 msg/s by the controller, and the reliability of each topic can reach 100% in the later stages.

no controller

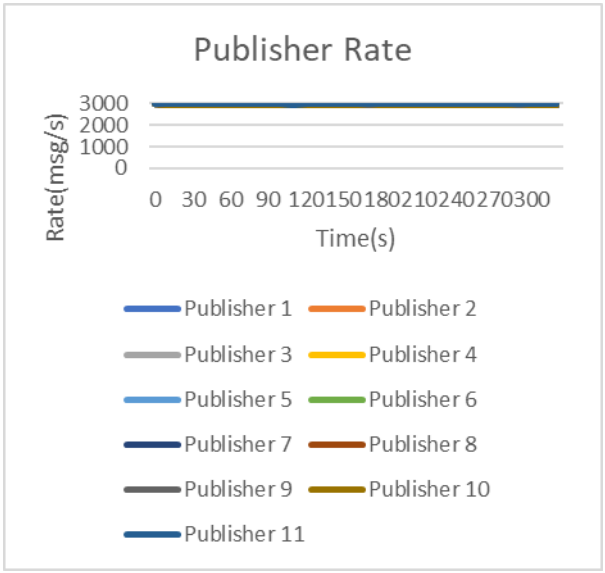

Figure 4 Publisher Rate-3000msg/s-Plan A-with controller

controller

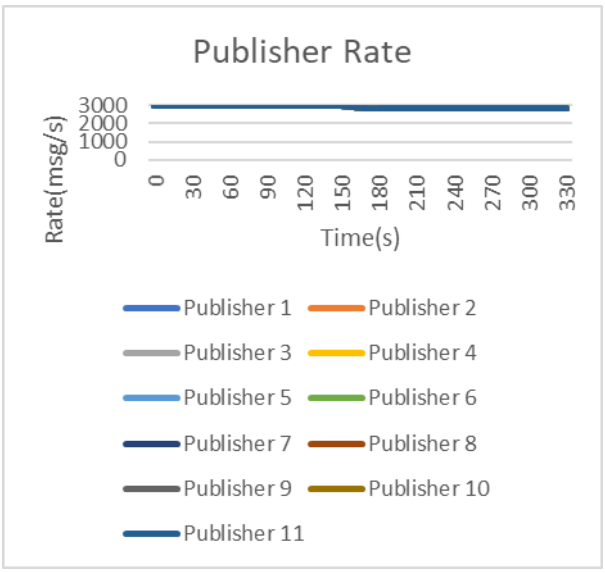

Figure 5 Publisher Rate-3000msg/s-Plan A-without controller

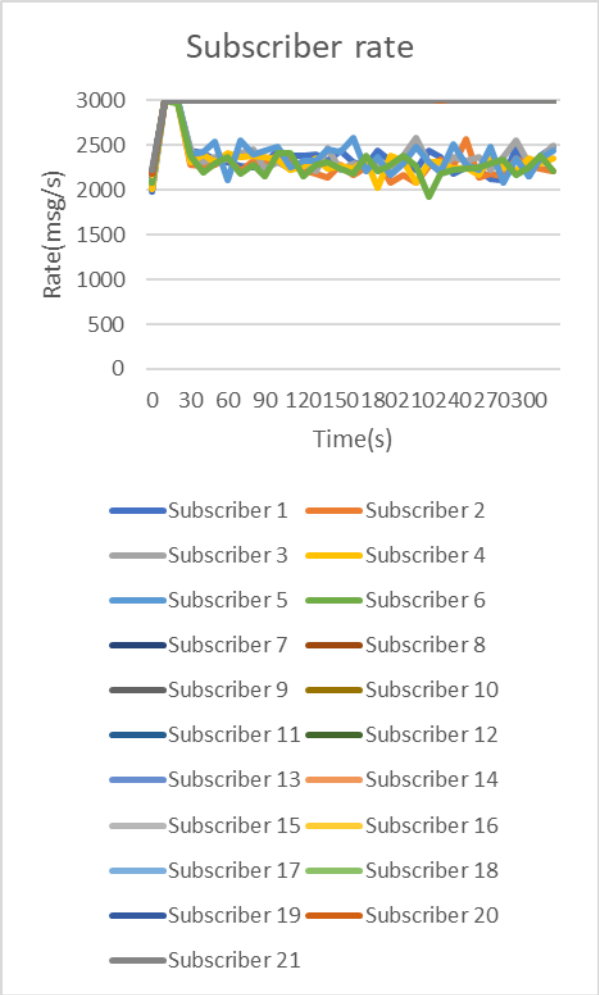

Figure 6 Subscriber Rate-3000msg/s -Plan A-with controller

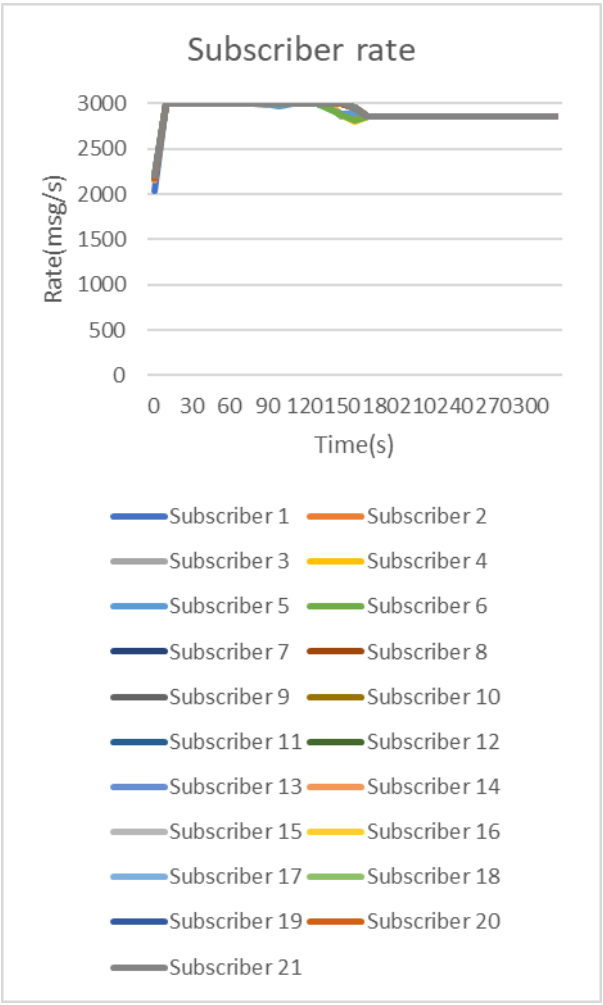

Figure 7 Subscriber Rate-3000msg/s-Plan A-without controller

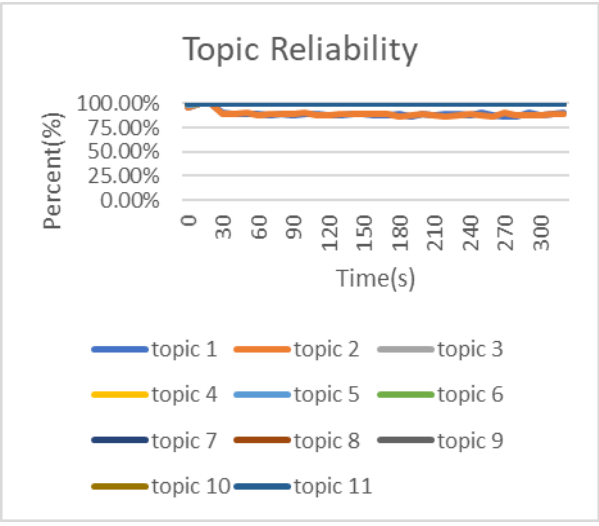

Figure 8 Topic reliability-3000msg/s-Plan A-with controller

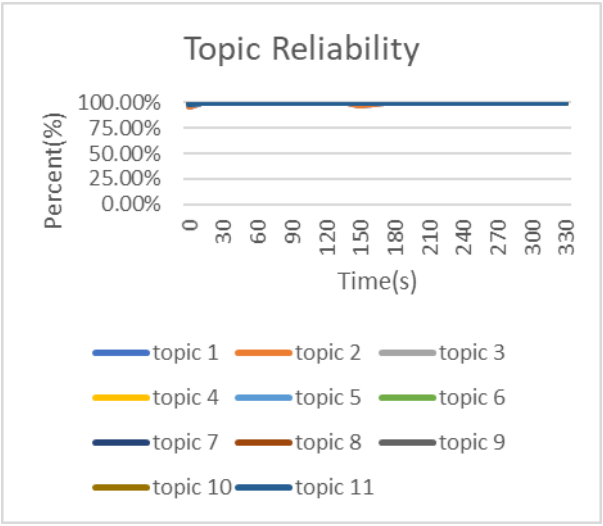

Figure 9 Topic reliability-3000msg/s-Plan A-without controller

2. Plan B

In Plan B, both host2 and host3 demonstrated good reception capabilities under low workload conditions, with only a slight improvement when the controller was activated. The host2 reception speed ranges from 2,777 msg/s to 2,999 msg/s, and the minimum ratio is greater than 0.97. Therefore, the controller determines that host2 is in a stable state and does not need to change the transmission speed.

no controller

Figure 10 Publisher Rate-3000msg/s-Plan B-without controller

controller

Figure 11 Publisher Rate-3000msg/s-Plan B-with controller

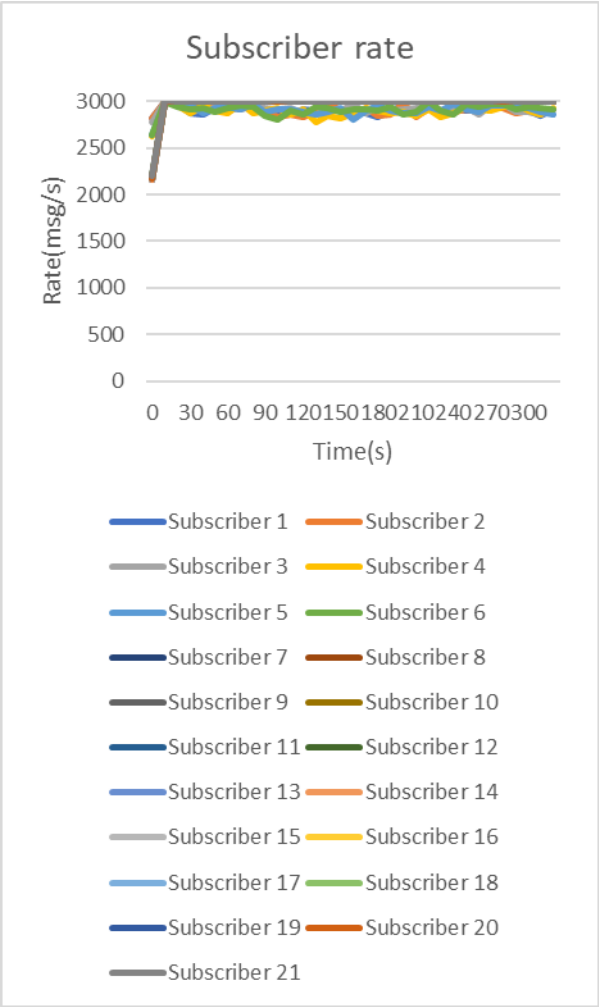

Figure 12 Subscriber Rate-3000msg/s-Plan B-without controller

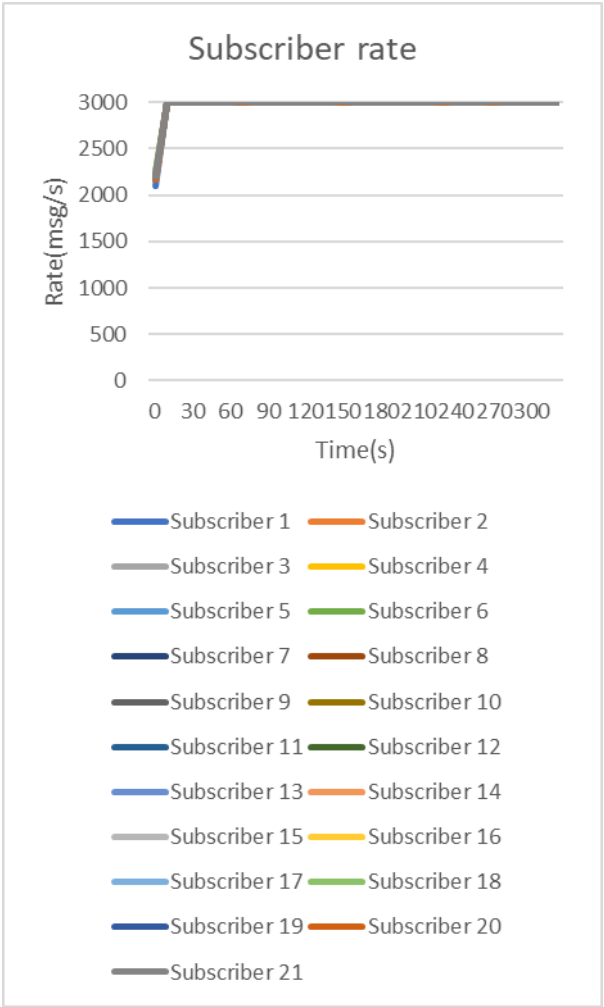

Figure 13 Subscriber Rate-3000msg/s-Plan B-with controller

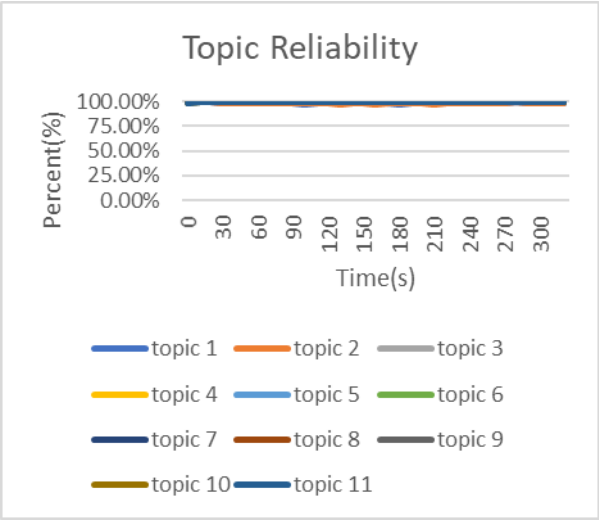

Figure 14 Topic reliability-3000msg/s-Plan B-without controller

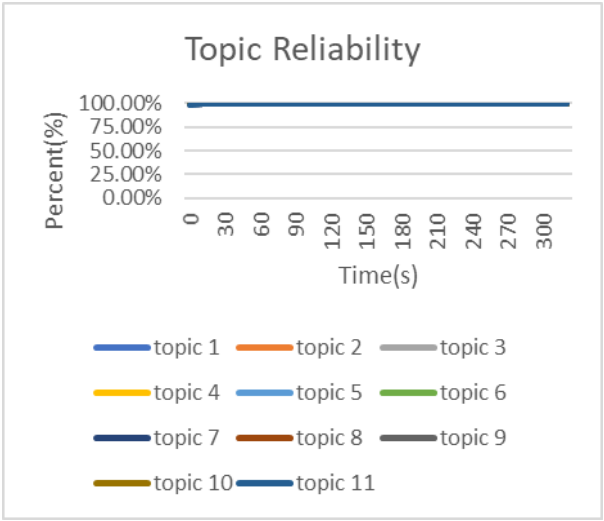

Figure 15 Topic reliability-3000msg/s-Plan B-with controller

3. Plan C

In Plan C, both host2 and host3 demonstrated good reception capabilities under low workload conditions, with only a slight improvement when the controller was activated.

no controller

Figure 16 Publisher Rate-3000msg/s-Plan C-without controller

controller

Figure 17 Publisher Rate-3000msg/s-Plan C-without controller

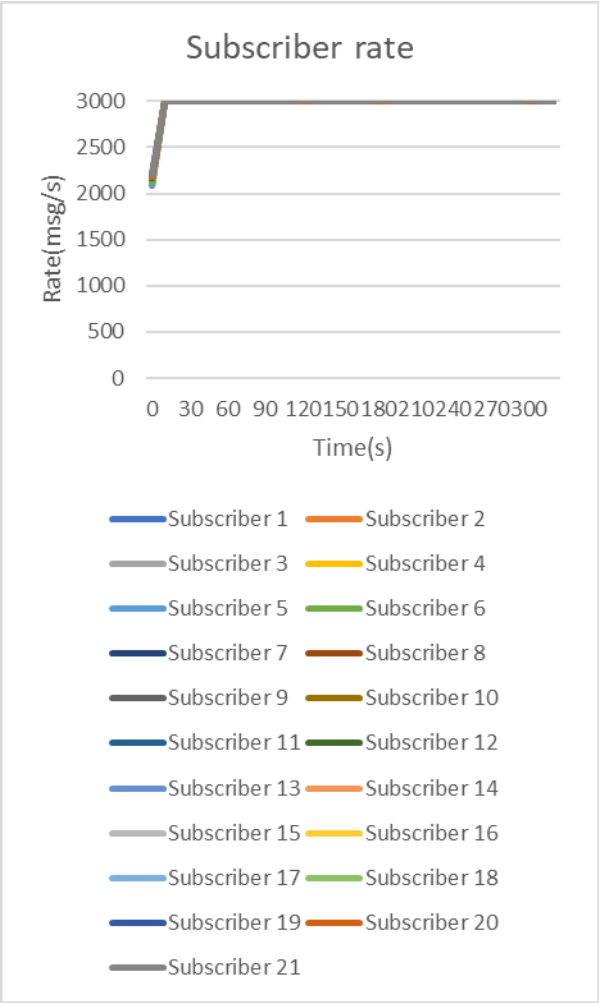

Figure 18 Subscriber Rate-3000msg/s-Plan C-without controller

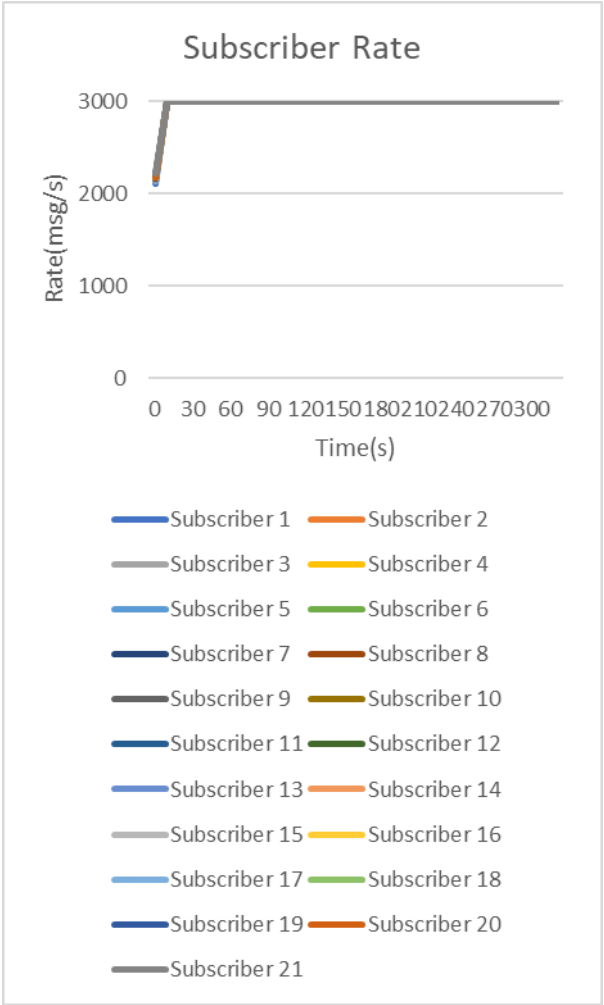

Figure 19 Subscriber Rate-3000msg/s-Plan C-with controller

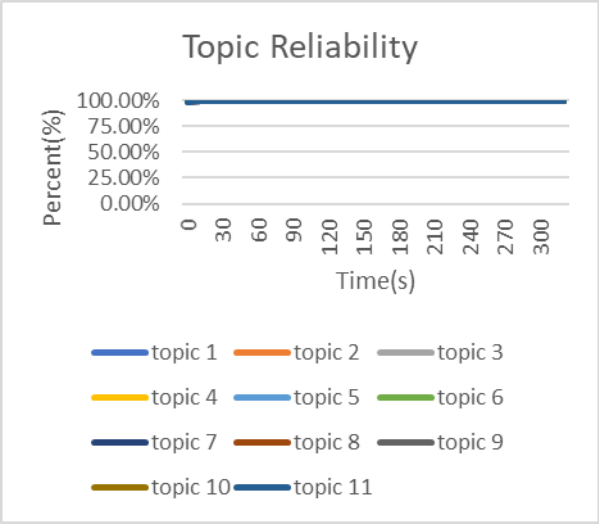

Figure 20 Subscriber Rate-3000msg/s-Plan C-without controller

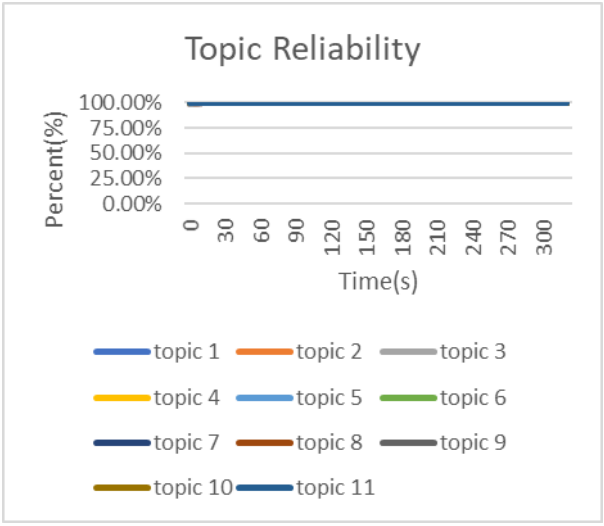

Figure 21 Subscriber Rate-3000msg/s-Plan C-with controller

5-3-2 Medium Workload-5000msg/s

Table 11 Host2-Medium workload

HOST2-5000msg/s		**no controller**		**controller**		
Plan	Expected	Received	System Reliability	Received	System Reliability	**Improve**
A	6,000,000	618,984	10.30%	4,814,259	80.20%	**69.90%**
B	6,000,000	4,253,707	70.90%	5,363,924	89.30%	**18.40%**
C	6,000,000	4,410,582	73.50%	5,480,100	91.30%	**17.80%**

Table 12 Host3- Medium workload

HOST3-5000msg/s		**no controller**		**controller**		
Plan	Expected	Received	System Reliability	Received	System Reliability	**Improve**
A	15,000,000	14,791,139	98.00%	14,890,501	99.30%	**1.30%**
B	15,000,000	14,792,774	98.60%	14,884,229	99.20%	**0.60%**
C	15,000,000	14,788,927	98.50%	14,934,173	99.60%	**1.10%**

In medium workloads, the reception capacity of host2 had a significant impact, but host3 still maintained good reception capacity. In Plan A, the reception capacity of host3 was much lower than that of Plan B and Plan C. The controller played a greater role in this case.

1. Plan A

In Plan A, the reception speed of host2 was between 370msg/s and 550msg/s. Meanwhile, host3 maintained good reception capability. The controller adjusted the transmission speed to 2024msg/s. The system reliability of host2 increased from 10.3% to 80.2%, which is a 69.9% improvement. host3 only saw a slight improvement. This is because of its original good reception capability.

no controller

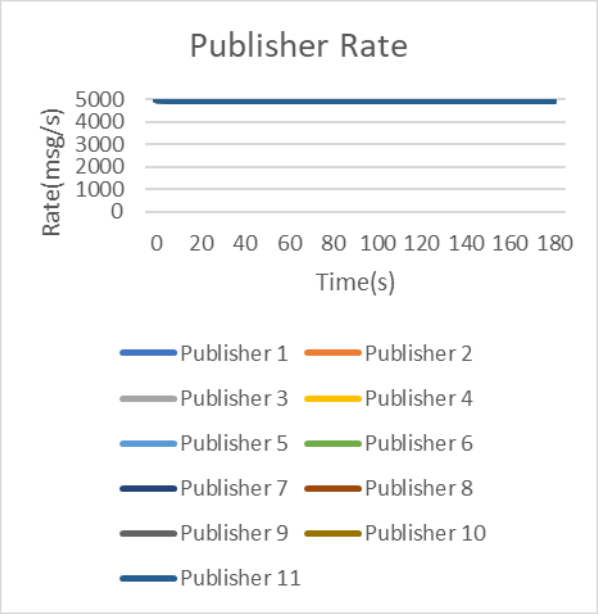

Figure 22 Publisher Rate-5000msg/s- -Plan A- with controller

controller

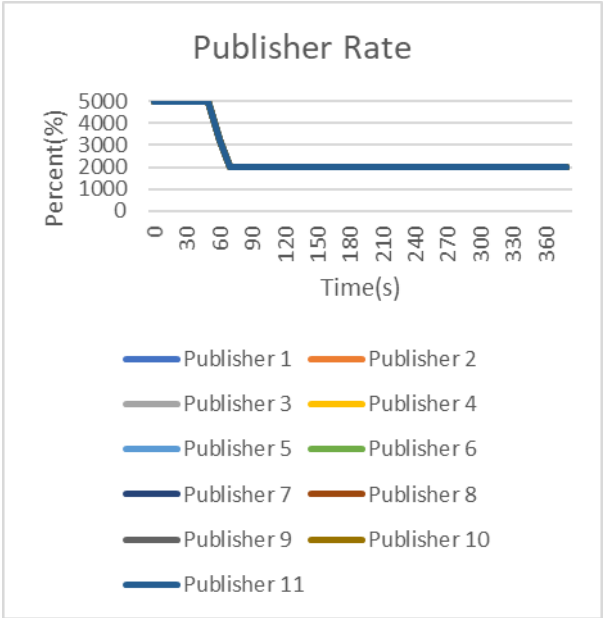

Figure 23 Publisher Rate-5000msg/s-Plan A- without controller

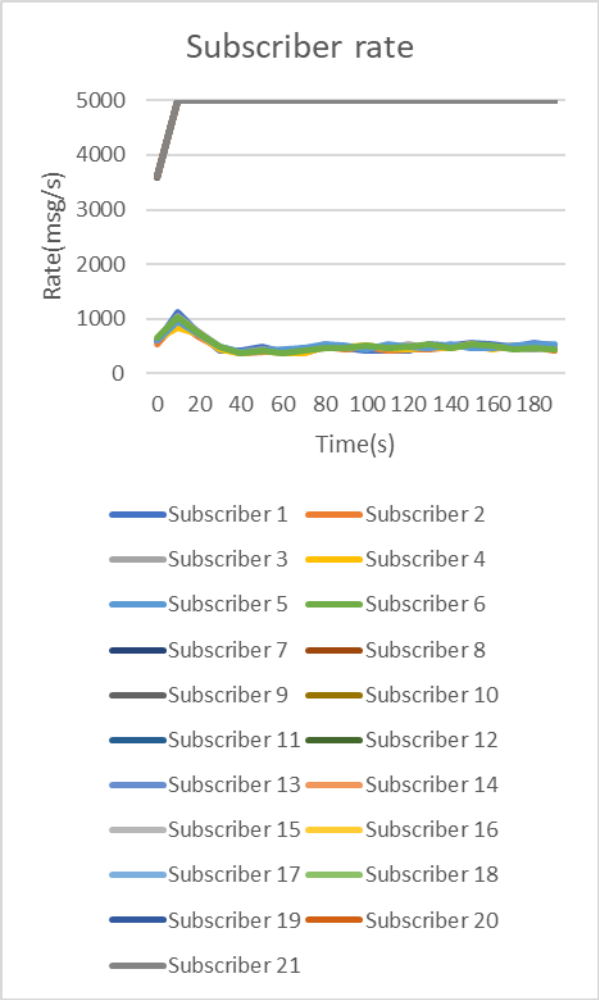

Figure 24 Subscriber Rate-5000msg/s- -Plan A-without controller

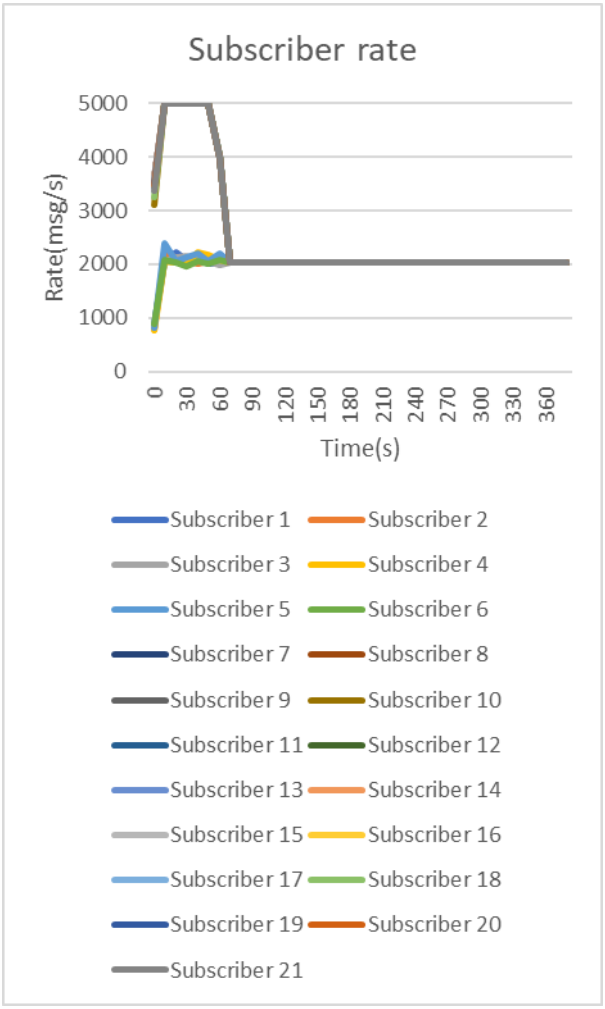

Figure 25 Subscriber Rate-3000msg/s- -Plan A-with controller

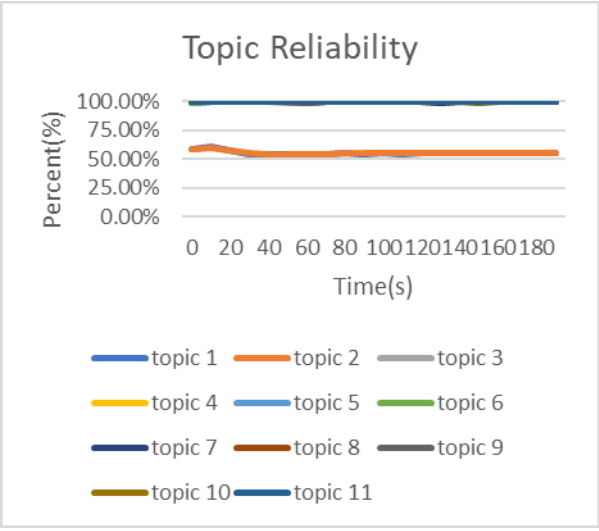

Figure 26 Topic reliability-5000msg/s-Plan A-without controller

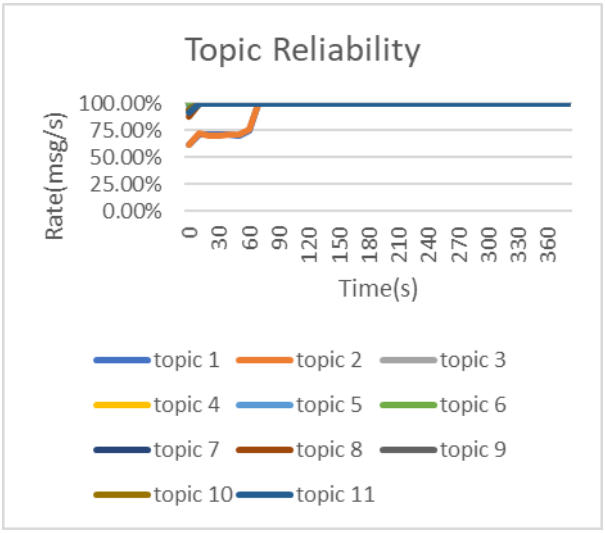

Figure 27 Topic reliability-5000msg/s-Plan A-with controller

2. Plan B

In Plan B, the reception speed of host2 was between 3,300msg/s and 3,700msg/s, while host3 maintained good reception capability. Once the controller had been started, the transmission speed was gradually adjusted to 2,949msg/s. Host2's reliability improved by 18.4%, whereas host3, which already had good reception capability, improved only slightly.

no controller

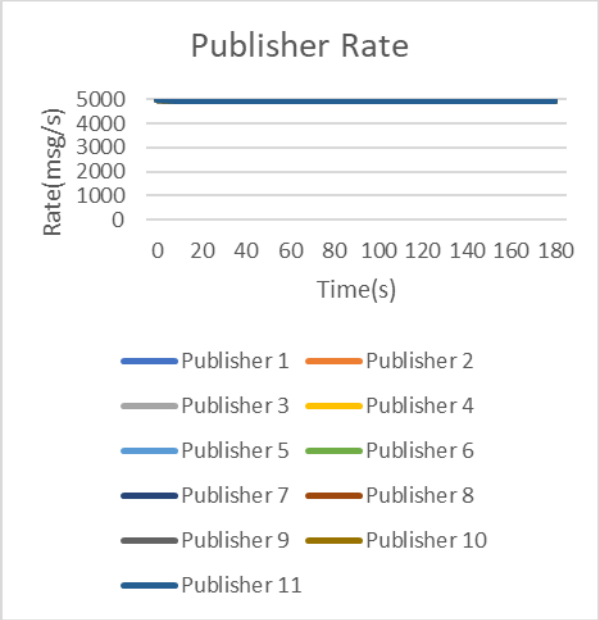

Figure 28 Publisher Rate-5000msg/s-Plan B-without controller

controller

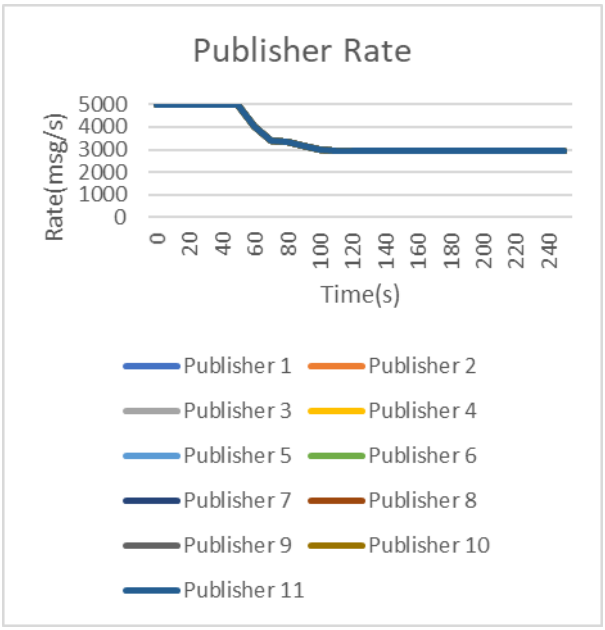

Figure 29 Publisher Rate-5000msg/s-Plan B-with controller

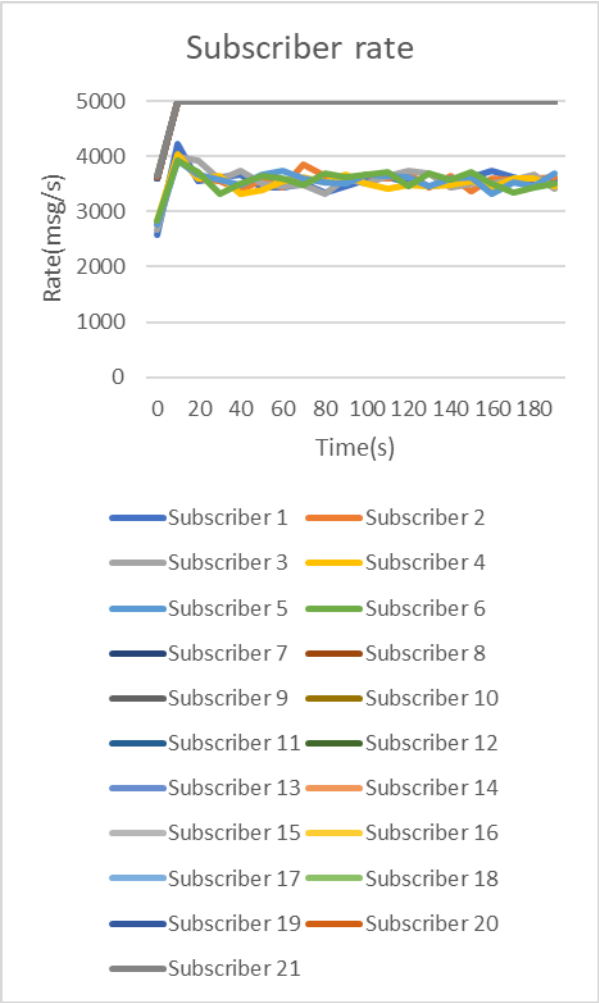

Figure 30 Subscriber Rate-3000msg/s -Plan B-without controller

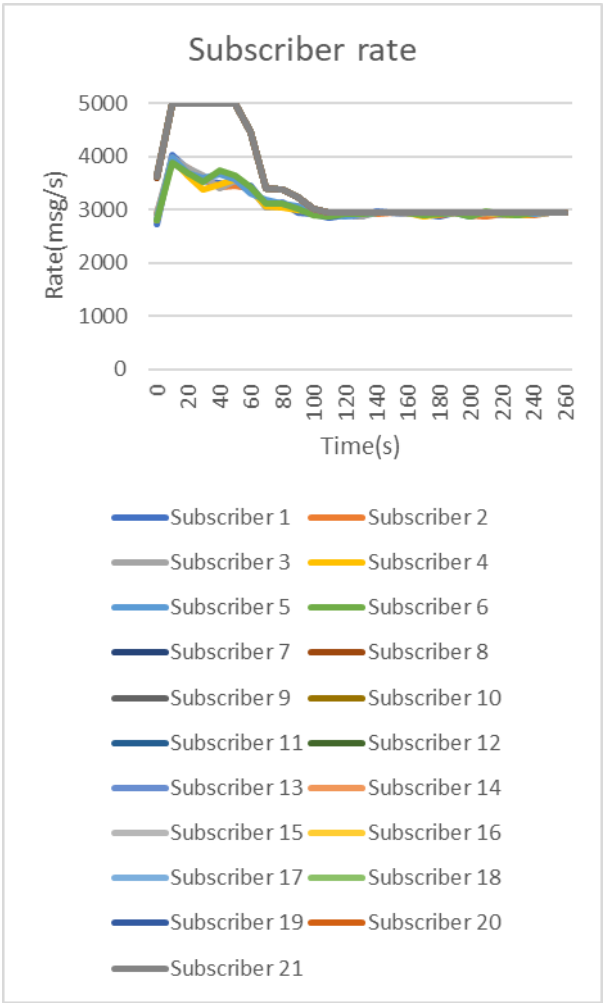

Figure 31 Subscriber Rate-3000msg/s-Plan B-with controller

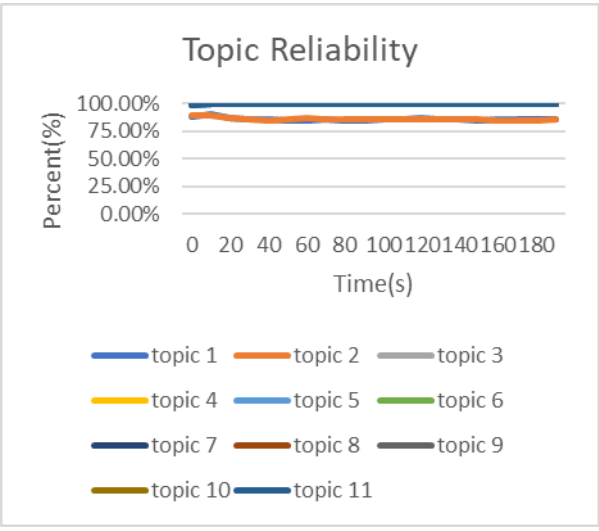

Figure 32 Topic reliability-5000msg/s-Plan B-without controller

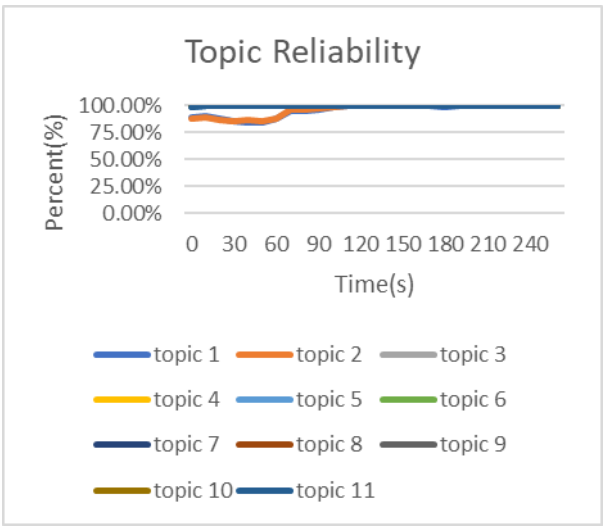

Figure 33 Topic reliability-5000msg/s-Plan B-with controller

3. Plan C

In Plan C, the reception speed of host2 was between 3,400msg/s and 4,000msg/s, while host3 maintained good reception capability. Once the controller had been started, the transmission speed was gradually adjusted to 3,300msg/s. host2's reliability improved by 17.8%, while host3's improved by only 1.1%, despite already having good reception capability.

no controller

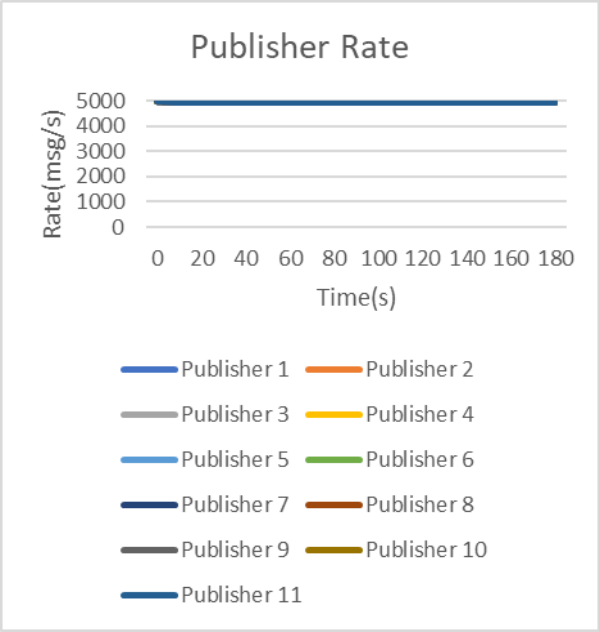

Figure 34 Topic reliability-5000msg/s-Plan C- without controller

controller

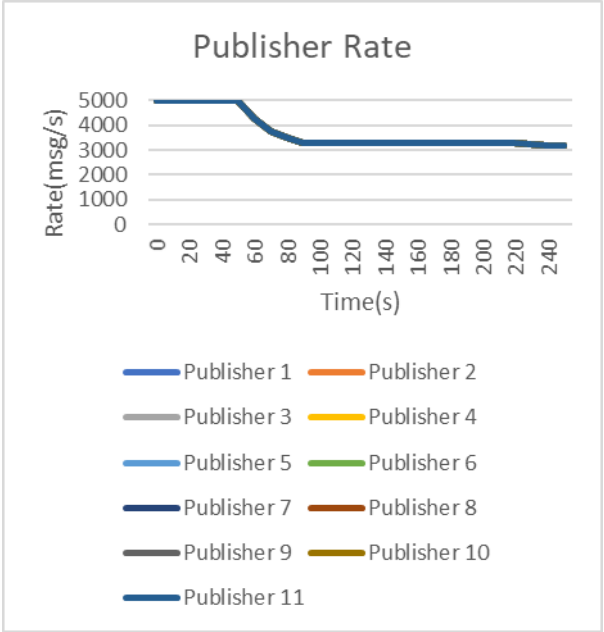

Figure 35 Topic reliability-5000msg/s-Plan C- with controller

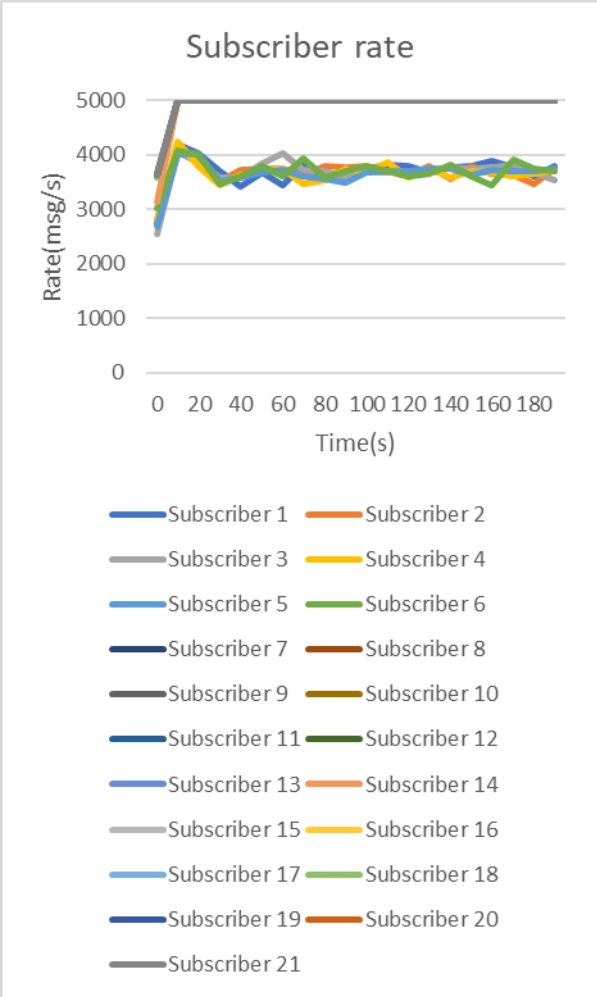

Figure 36 Subscriber Rate-5000msg/s-Plan C- without controller

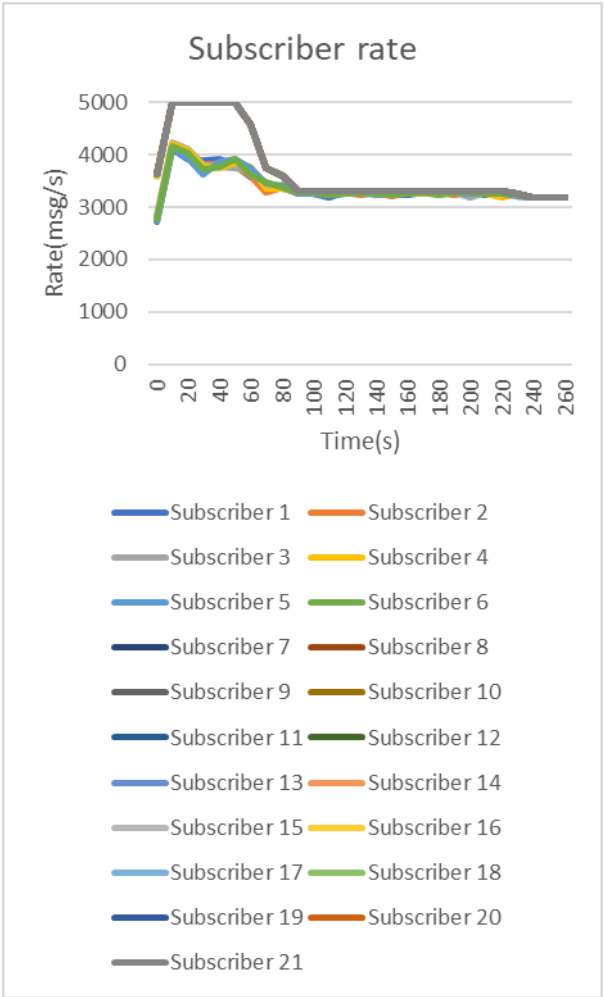

Figure 37 Subscriber Rate-5000msg/s-Plan C- with controller

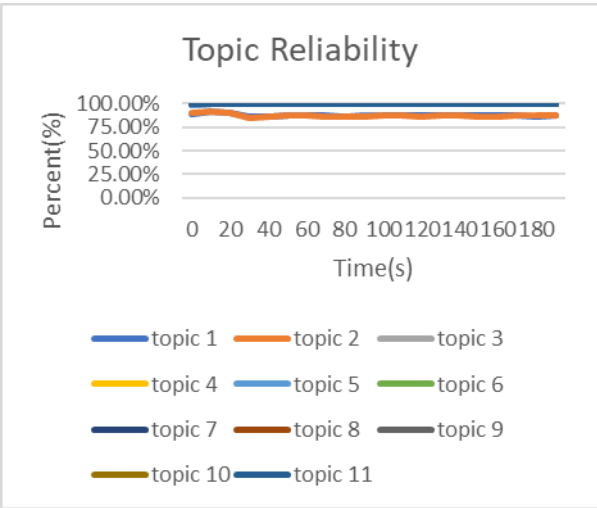

Figure 38 Topic reliability-5000msg/s-Plan C- without controller

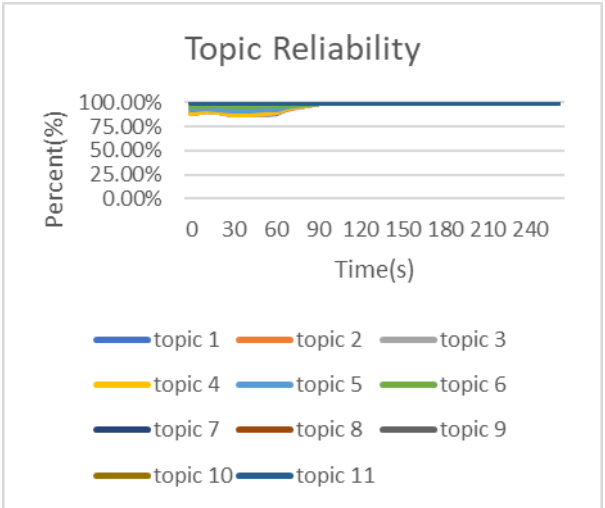

Figure 39 Topic reliability-5000msg/s-Plan C- with controller

5-3-1 High Workload-7000msg/s

Table 13 Host2- High workload

HOST2-7000msg/s		**no controller**		**controller**		
Plan	Expected	Received	System Reliability	Received	System Reliability	**Improve**
A	6,000,000	23,942	0.40%	3,394,759	56.58%	**56.18%**
B	6,000,000	3,138,987	52.32%	4,737,098	78.95%	**26.64%**
C	6,000,000	3,358,766	55.98%	4,570,210	76.17%	**20.19%**

Table 14 Host3- High workload

HOST3-7000msg/s		**no controller**		**controller**		
Plan	Expected	Received	System Reliability	Received	System Reliability	**Improve**
A	15,000,000	13,664,055	91.09%	14,895,866	99.31%	**8.21%**
B	15,000,000	14,117,788	94.12%	14,602,393	97.35%	**3.23%**
C	15,000,000	14,234,108	94.89%	14,473,861	96.49%	**1.60%**

Even under high workload conditions, host3 maintains good reception capability. Under the demanding Plan A, the reception speed can reach almost 6,500msg/s. However, host2 is significantly impacted, particularly under Plan A, when the system is almost disabled. At this point, the controller activates the protection mechanism to improve performance.

1. Plan A

In Plan A, the requirement of rel_qos caused a sharp drop in host2 performance, with an acceptance speed of less than 20 msg/s. Meanwhile, host3's reception speed ranged from 6315msg/s to6989 msg/s and both hosts were unstable. The controller then activated its protection mechanism, adjusting the sending rate to 100 msg/s, and host3's reliability improved by 56.18%, rising from 0.4% to 56.58%.

no controller

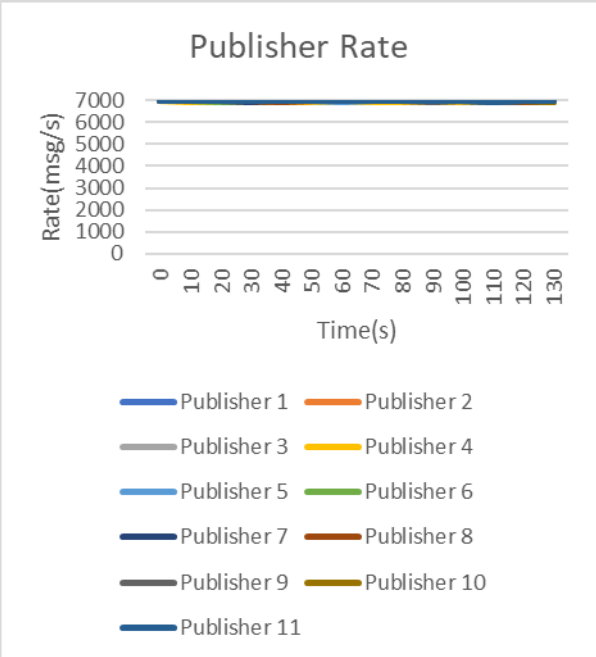

Figure 40 Publisher Rate-7000msg/s- -Plan A- without controller

controller

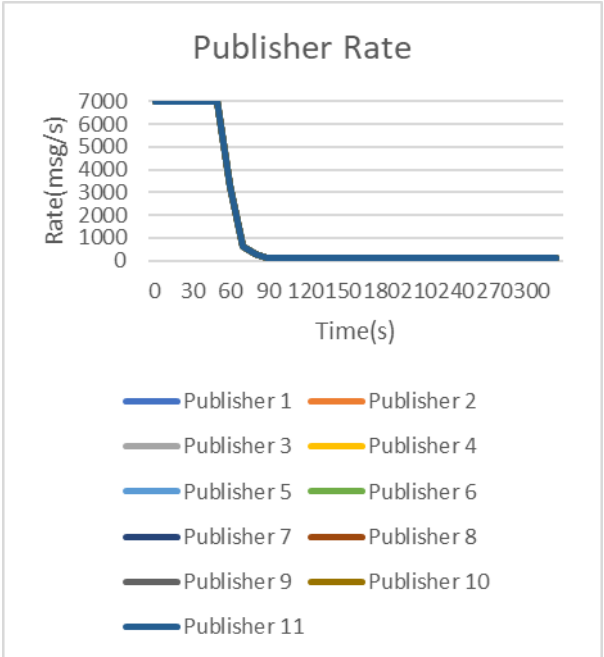

Figure 41 Publisher Rate-7000msg/s- -Plan A- with controller

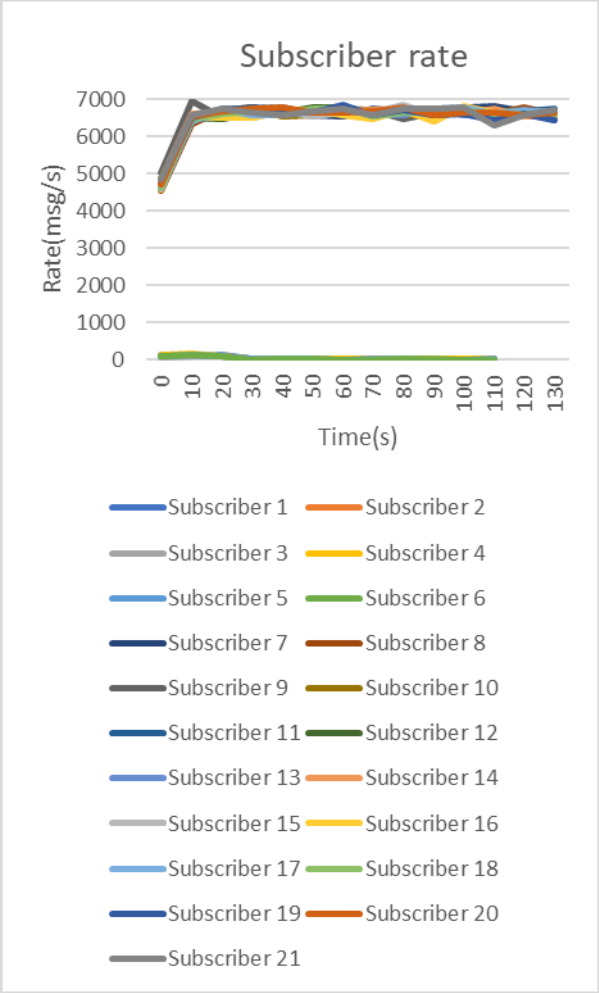

Figure 42 Subscriber Rate-7000msg/s- -Plan A-with controller

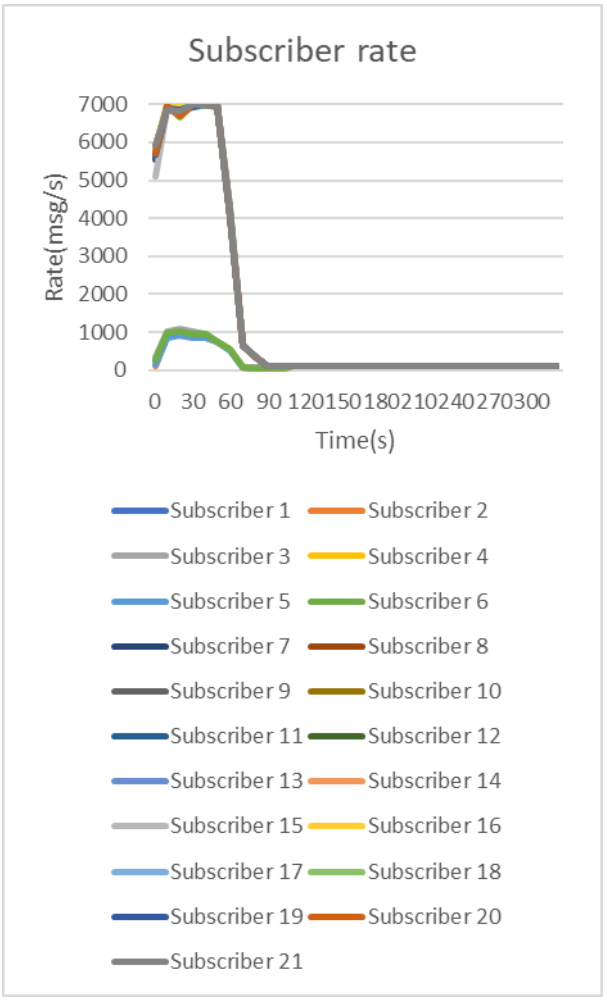

Figure 43 Subscriber Rate-7000msg/s- -Plan A-without controller

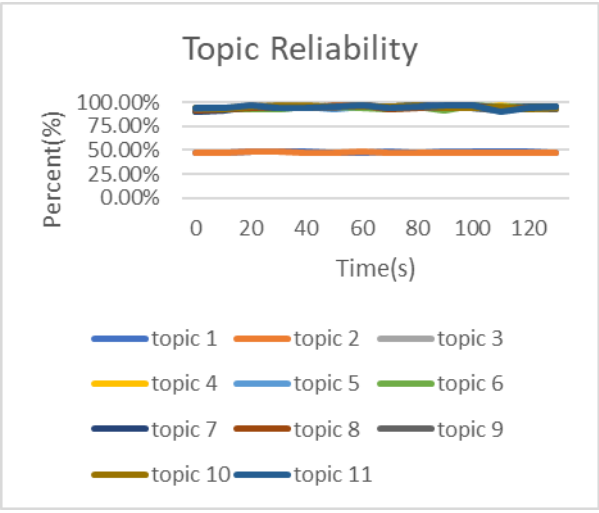

Figure 44Topic reliability-7000msg/s-Plan A-with controller

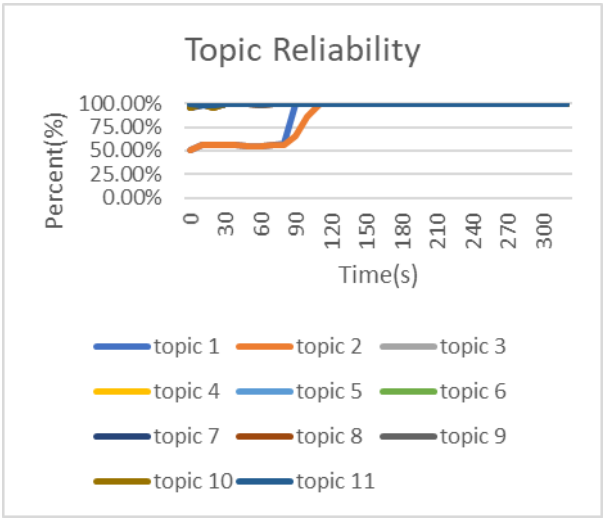

Figure 45 Topic reliability-7000msg/s-Plan A-without controller

2. Plan B

In Plan B, the observed receiving rate of host2 is between 3414msg/s and 4086msg/s, host3 is between 6717msg/s and 6981msg/s respectively due to the characteristics of reliable QoS. Both hosts were unstable. The controller was started and the transmission speed gradually adjusted to 3,225msg/s. The reliability of host3 improved by 26.64%.

no controller

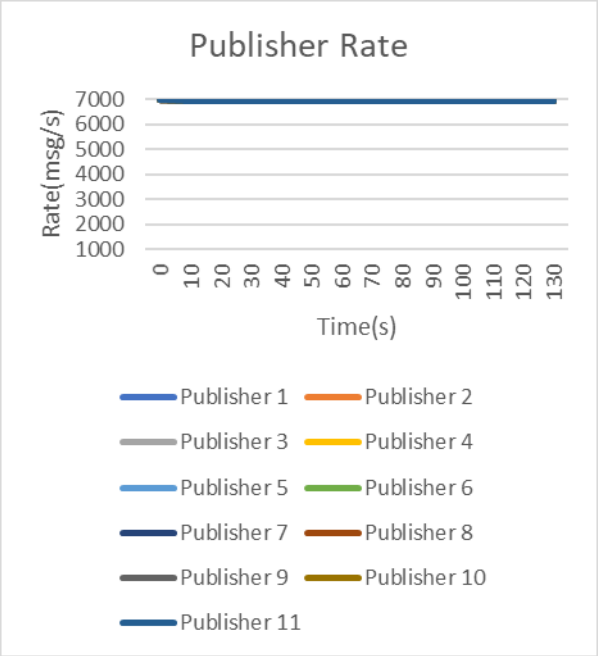

Figure 46 Publisher Rate-7000msg/s-Plan B-without controller

controller

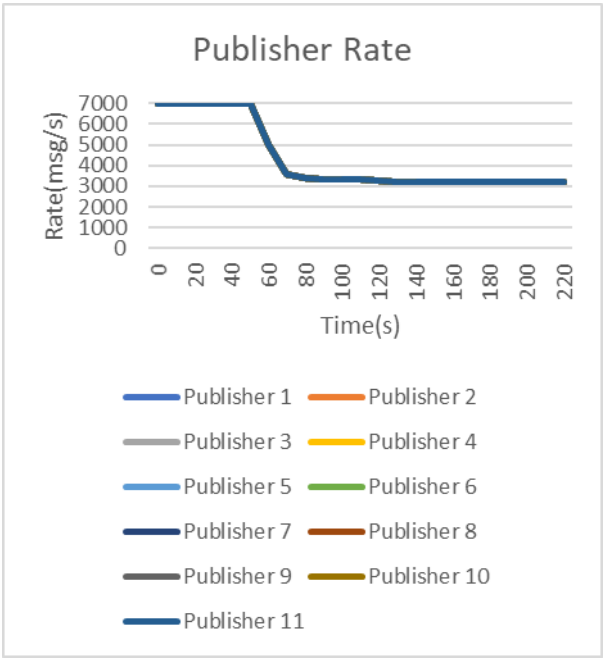

Figure 47 Publisher Rate-7000msg/s-Plan B-with controller

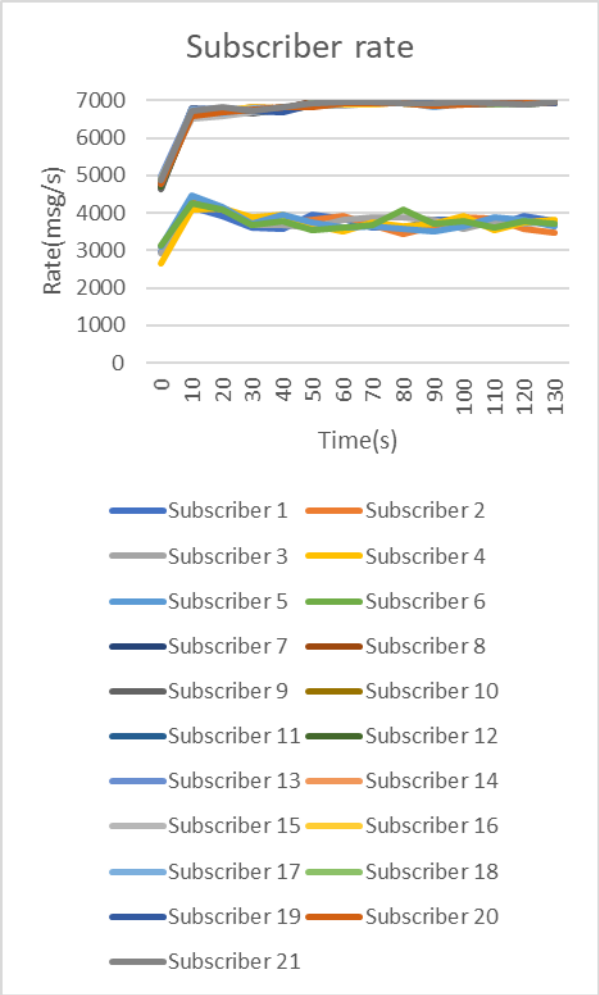

Figure 48 Subscriber Rate-7000msg/s -Plan B-without controller

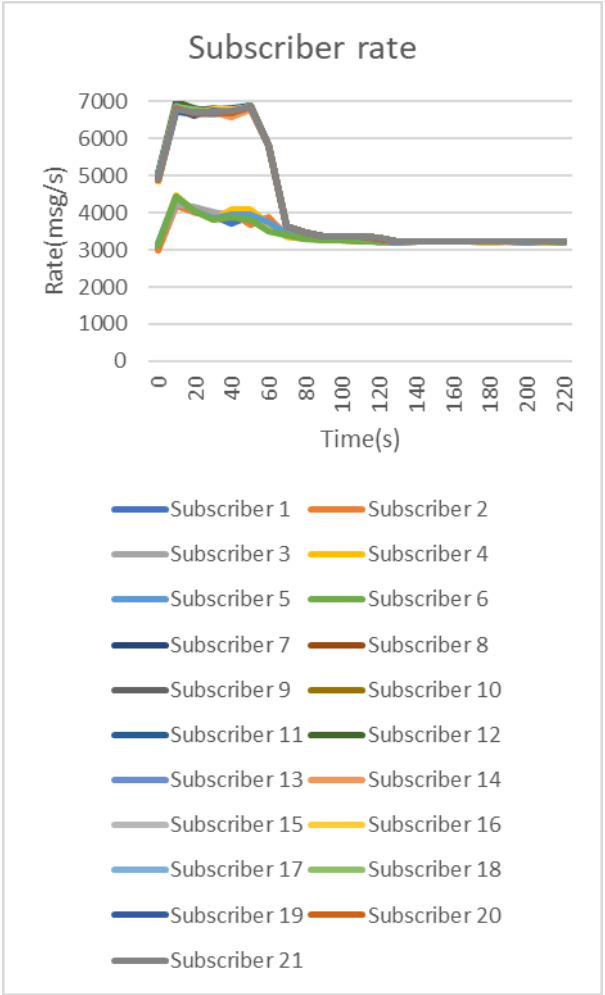

Figure 49 Subscriber Rate-7000msg/s -Plan B-with controller

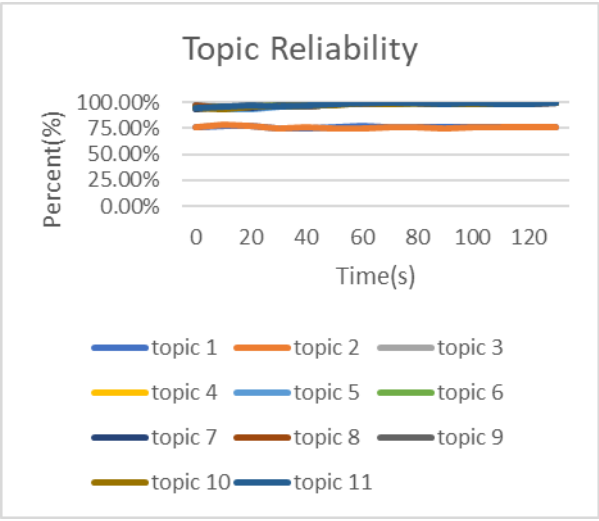

Figure 50 Topic reliability-7000msg/s-Plan B-without controller

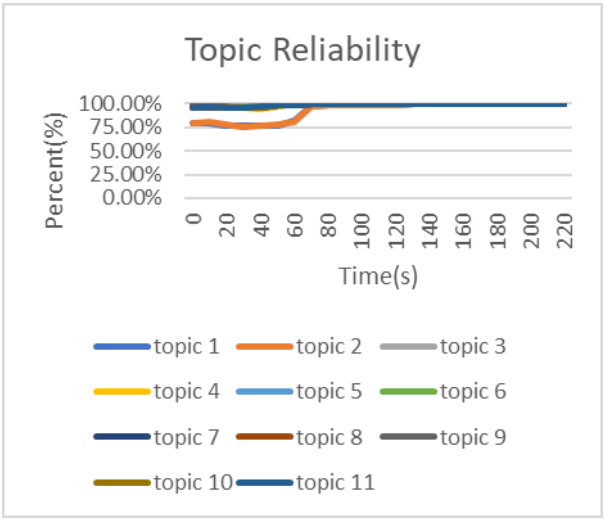

Figure 51 Topic reliability-7000msg/s-Plan B-with controller

3. Plan C

In Plan C, the reception rate for Host 2 ranges from 3,662msg/s to 4,976 msg/s. For host 3, the reception rate ranges from 6,825 msg/s to 6,982 msg/s. Once the controller has been started, the sending rate is adjusted to 3,135 msg/s. The reliability of some topics increases from 75% to 100%. Host 2 shows significant improvement, with its reliability increasing by 20.19%, whereas host 3's reliability improves only slightly, by 1.6%.

no controller

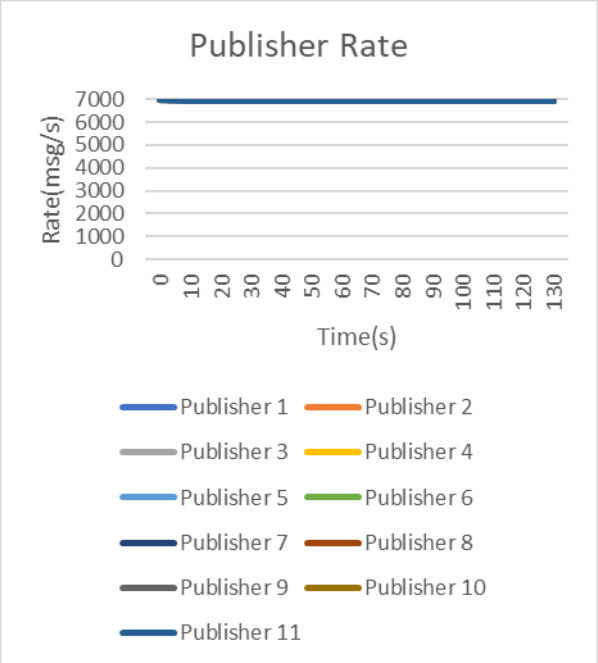

Figure 52 Topic reliability-7000msg/s-Plan C- without controller

controller

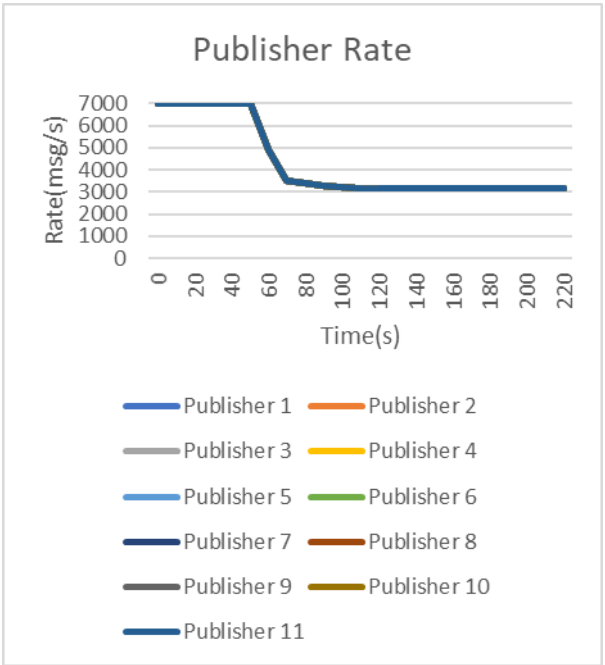

Figure 53 Topic reliability-7000msg/s-Plan C- with controller

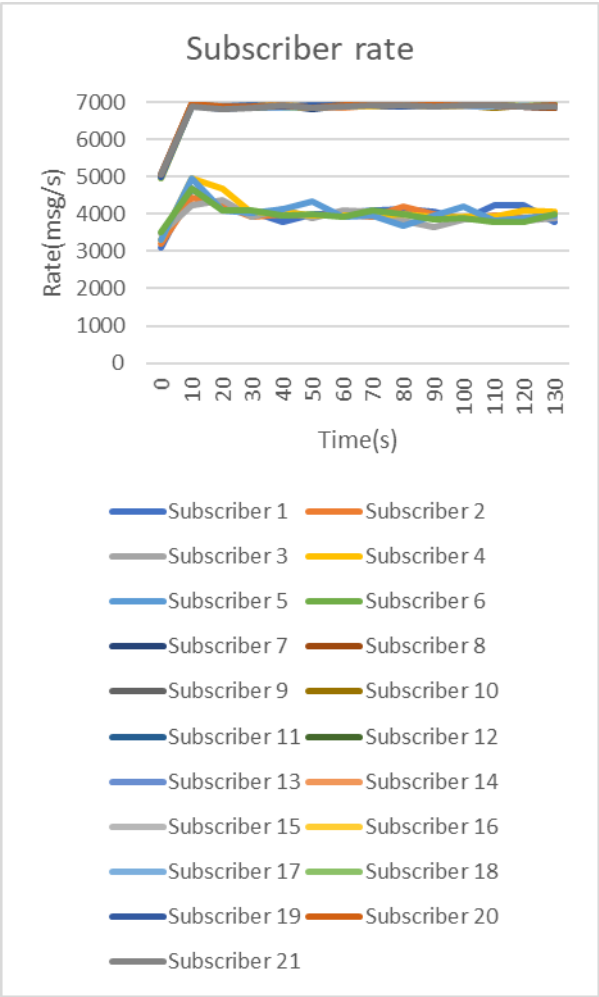

Figure 54 Subscriber Rate-7000msg/s-Plan C- without controller

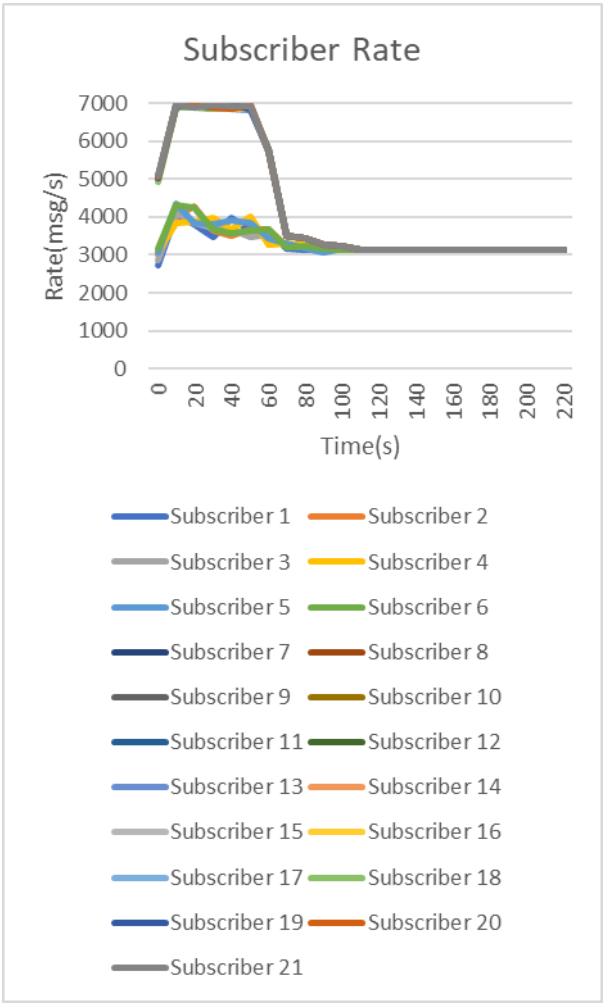

Figure 55 Subscriber Rate-7000msg/s-Plan C- with controller

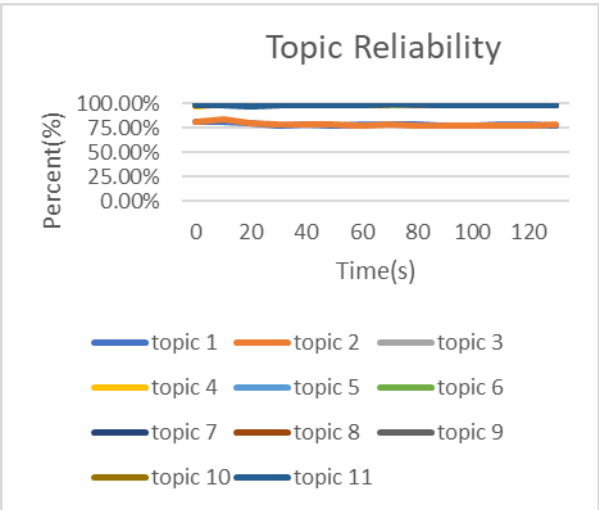

Figure 56 Topic reliability-7000msg/s-Plan C- without controller

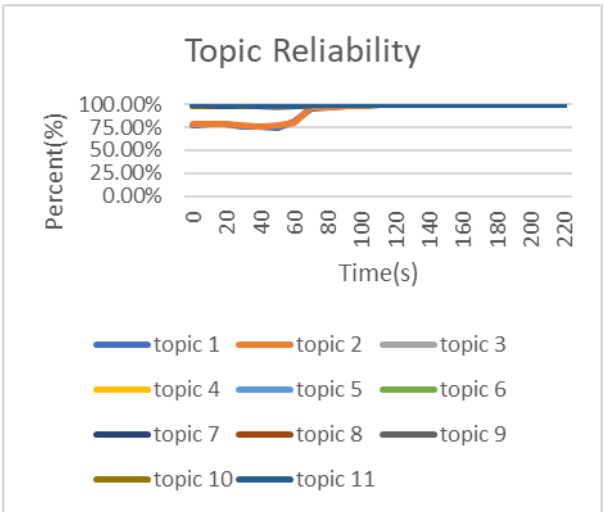

Figure 57 Topic reliability-7000msg/s-Plan C- without controller

In host2, the reliable QoS performs poorly. Plan A achieves an reliability of just 10.30% under a medium load. When faced with a high workload, Plan A experiences severe packet loss without the controller, achieving an reliability of just 0.4%. The RELIABLE QoS requires extensive buffer management and Windows' memory allocation strategy can cause bottlenecks. The reception rate was only 0.4% before the controller was enabled, but this improved significantly afterwards, by 56.18%. This indicates that traditional fixed transmission rate methods severely impact DDS performance when the system is overloaded or the network is congested, whereas dynamic rate adjustment can significantly mitigate this issue.

In host3, system reliability remains above 95% across all load levels. Even under the high-load condition of 7,000 msg/s, minimum system reliability still reaches 91.09%. Host3 demonstrates excellent load tolerance, maintaining stable performance even under high loads. The QoS handling mechanism is robust, with minimal differences between the various schemes. Both RELIABLE and BEST_EFFORT modes are stable. The base reception rate of host2 is already high. The controller's improvement effect is small, with only a slight increase in reception reliability.

Chapter VI. Conclusion

This experiment validated the effectiveness of the dynamic transmission rate control mechanism in enhancing the performance of DDS systems across various QoS policies and transmission rates. Notably, in scenarios involving high loads and low performance at the receiving end, the controller can significantly enhance system availability and reception reliability. This makes it highly valuable for DDS applications in multi-host, heterogeneous network environments.

In the future, machine learning or adaptive control algorithms could be introduced to dynamically predict the optimal transmission rate based on historical data and real-time network state (e.g., RTT and packet loss rate trends), rather than relying solely on the current reception rate for adjustments. Currently, the system restricts bandwidth allocation by topic, but we may consider removing these restrictions in the future.

References

[1] G. Pardo-Castellote, "Omg data-distribution service: Architectural overview," in *23rd International Conference on Distributed Computing Systems Workshops, 2003. Proceedings.*, 2003, pp. 200-206: IEEE.

[2] Real Time Innovations. (2025). *Data Distribution Service (DDS) Community-RTI Connext Users*. Available: https://community.rti.com/documentation

[3] eProsima. (2025). *eProsima Fast DDS*. Available: https://fast-dds.docs.eprosima.com/en/latest/

[4] OpenDDS Foundation. (2025). *OpenDDS*. Available: https://opendds.readthedocs.io/en/latest-release/

[5] Eclipse Foundation. (2025). *Eclipse Cyclone DDS*. Available: https://cyclonedds.io/docs/

[6] ADLINK Technology Inc. (2025). *Vortex OpenSplice DDS*. Available: https://www.adlinktech.com/en/dds-community-software-evaluation

[7] Twin Oaks Computing, Inc. (2025). *CoreDX DDS*. Available: https://www.twinoakscomputing.com/coredx/documentation

[8] S. Jeong, T. Ga, I. Jeong, and J. Choi, "Behavior tree driven multi-mobile robots via data distribution service (DDS)," in *2021 21st International Conference on Control, Automation and Systems (ICCAS)*, 2021, pp. 1633-1638: IEEE.

[9] M. L. Gambo, A. Danasabe, B. Almadani, F. Aliyu, A. Aliyu, and E. J. R. Al-Nahari, "A Systematic Literature Review of DDS Middleware in Robotic Systems," *Robotics,* vol. 14, no. 5, p. 63, 2025.

[10] Y. Park, D. Lim, D. Min, S. A. Kim, "Research on Design of DDS-based Conventional Railway Signal Data Specification for Real-time Railway Safety Monitoring and Control," *Journal of the Korea Institute of Information and Communication Engineering*, vol. 20, no. 4, pp. 739-746, 2016.

[11] J. So, K.-H. Shin, and J. Ahn, "A Study on DDS (Data Distribution Service) Application for Real-time Monitoring and Control in Operation Console of the Railway Safety Control Platform," *Journal of The Korean Society For Urban Railway,* vol. 6, no. 4, pp. 279-286, 2018.

[12] M. Essers and T. H. Vaneker, "Evaluating a prototype approach to validating a DDS-based system architecture for automated manufacturing environments" *Procedia CIRP,* vol. 25, pp. 385-392, 2014.

[13] B. Almadani and S. M. Mostafa, "IIoT based multimodal communication

model for agriculture and agro-industries," *IEEE Access,* vol. 9, pp. 10070-10088, 2021.

[14] G. Yoon, S. Lee, and H. Choi, "Qos optimizer," in *2016 International conference on platform technology and service (PlatCon)*, 2016, pp. 1-5: IEEE.

[15] R. S. Auliya, C.-C. Chen, P.-R. Lin, D. Liang, and W. J. Wang, "Optimization of message delivery reliability and throughput in a DDS-based system with per-publisher sending rate adjustment," *Telecommunication Systems,* vol. 84, no. 2, pp. 235-250, 2023.

[16] Z. Meng *et al.*, "Enabling high quality {Real-Time} communications with adaptive {Frame-Rate}," in *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, 2023, pp. 1429-1450.

[17] B. Almadani and S. Abudalfa, "QoS adaptation for publish/subscribe middleware in real-time dynamic environments," *International Arab Journal of Information Technology (IAJIT),* vol. 14, no. 2, 2017.

[18] A. Baunthiyal, M. G. Rakesh, A. K. Jain, and E. Technology, "Criteria Set for Evaluation of different DDS Distributions," *International Journal for Research in Applied Science and Engineering Technology,* vol. 9, no. 1, pp. 119-128, 2021.

[19] Y. Fu, L. Hao, and D. Guo, "Application research of distributed simulation system based on data distribution," in *2019 IEEE international conference on unmanned systems and artificial intelligence (ICUSAI)*, 2019, pp. 268-273: IEEE.